

Performance Methodology

How to move from signals to solutions. The vocabulary that decides what you measure, the posture that decides what you see, the methods worth running — and the statistics that will lie to you if you let them.

1

The words decide the measurement

Latency, utilization's two meanings, saturation, caching, the knee point.

2

Where you stand decides what you see

Two perspectives, three anti-methods, and the procedures that replace them.

3

The methods worth running

USE, RED, workload characterization, drill-down, latency analysis, and the ones people skip.

4

The numbers will lie to you

Scalability models, queueing theory, averages, distributions, outliers.

Print double-sided, short-edge bind, A4 · 100% scale (no “fit to page”).

Answer key starts at Section 9 — keep a sheet of paper over it.

How to use this booklet

The loop, per section

- 1. Read the plain-version box first. If it doesn't land, the rest won't either.
- 2. Read the teaching. Say each boundary out loud: “X is *this*, its look-alike Y is *that*.”
- 3. Cover the page. Write the answer to **Q1** in the blank lines, in your own words, in full sentences. **Then** check the key.
- 4. Score yourself on completeness. Then do **Q2** and note whether it beat Q1.
- 5. Only then move on.

THE RULE THAT DOES MOST OF THE WORK

If you find yourself thinking “yes, I know this” — that is *recognition*, and it is not learning. The only evidence that counts is a written answer produced with the page covered.

WHAT MAKES THIS SUBJECT DIFFERENT FROM THE OTHERS

Most of what follows is not knowledge about a system; it is **procedure for approaching an unfamiliar one**. Learning the definition of a metric is easy and nearly useless on its own — the difficulty is knowing which metric to reach for, in what order, and when to stop. So when you meet a method here, do not just learn its steps: learn *which situation it is the answer to*, and what it costs you to run it. A method you can recite but cannot choose is worth nothing at three in the morning.

The spacing schedule

Review at **day 1, day 3, day 7, day 16, day 35**. On a pass, jump to the next interval. On a fail, reset that item to 1 day. Use the grid at the back; one row per item.

What's in here

1 — Learning goal and roadmap	orientation
2 — The words decide the measurement	teaching · questions · matrix
3 — Where you stand decides what you see	teaching · questions · matrix
4 — The methods worth running	teaching · questions · matrix
5 — The numbers will lie to you	teaching · questions · matrix
6 — Concept sketch + master matrix	consolidation
7 — Symptom → diagnosis → move · reverse mapping	working reference
8 — Numbers worth remembering · Glossary	reference
9 — Answer key	keep covered
10 — Spaced-review tracker · final quiz	workbook

Learning goal & roadmap

LEARNING GOAL — WHAT YOU SHOULD BE ABLE TO DO WHEN YOU CLOSE THIS

Handed an unfamiliar system that is “slow”, with no idea where to start, you can:

1. **Ask the six questions** that define the problem before you log in to anything.
2. **Choose a method** — and say which situation it is the answer to, rather than reaching for the tool you happen to know.
3. **Quantify what you find**, so you can rank it against everything else and know when to stop.
4. **Distrust the statistic** in front of you: name what an average hides, what a five-minute interval hides, and what a percentile will not tell you about a bimodal distribution.

The core model in one paragraph

You can learn every metric a system emits and still be unable to fix anything, because the hard part is not the definitions — it is the procedure for moving from signals to solutions. That procedure has four layers. **Vocabulary comes first**, because a word with two meanings will send you to the wrong place: utilization means two different things, and one of them stops being a warning exactly when you need it. **Then posture**: analysis can start from the resources or from the requests, and the perspective you inherit from whatever documentation you read is not the only one. **Then method**: a small number of procedures — check every resource, check every service, characterise the input, split the latency in half — that tell you where to start and what to do next, as opposed to the anti-methods, whose common signature is a hypothesis with no data behind it. **Then the numbers**, which are the last place you can still fool yourself: averages that describe nothing that happened, intervals that hide the seconds that mattered, and a mean response time that has already doubled once utilization passes 60%.

TENTH-GRADER VERSION — THE WHOLE THING AT ONCE

- Knowing what all the dials mean doesn't tell you which dial to look at. That's the whole problem.
- Some words are traps. “Busy all the time” and “can't take any more” sound the same and aren't.
- You can look at the machine's parts, or at the work being asked of it. Most people only ever learn one of those.
- The bad ways of investigating all look the same from outside: someone has a theory and no evidence for it.
- And when you finally have numbers, the numbers can still mislead you — especially averages.

The four parts, and why they're in this order

PART	WHAT IT COVERS	WHY HERE
1 The words decide the measurement	Latency, response time and service time; utilization's two incompatible definitions; saturation; throughput and IOPS; the knee point and scalability profiles; caching, hit ratios and warmth; load versus architecture; where tuning pays; when to stop.	Every later part depends on these being unambiguous. A term with two meanings quietly redirects an entire investigation.

2 Where you stand decides what you see	Resource analysis versus workload analysis; the three anti-methods and the tell they share; the ad hoc checklist; the problem statement; the scientific method; the diagnosis cycle; the tools method and its blind spot.	Before choosing a method you have to notice you were standing somewhere — and that the bad methods are not stupid, just untethered from data.
3 The methods worth running	USE and RED; workload characterization; drill-down analysis and Five Whys; latency analysis; event tracing; baseline statistics; static performance tuning; cache tuning; micro-benchmarking; the performance mantras.	The working core. Each is a small procedure that answers “where do I start” for a different shape of problem.
4 The numbers will lie to you	Scalability profiles; Amdahl's Law and the Universal Scalability Law; queueing theory, Little's Law and M/D/1 past 60%; resource limits and factor analysis; quantifying gains; averages, percentiles, coefficient of variation; multimodal distributions and outliers; monitoring patterns and visualizations.	The last place to go wrong. Having found something and measured it, this is where you decide whether the measurement means what you think.

“Then there was the man who drowned crossing a stream with an average depth of six inches.”

1

PART 1 OF 4

The words decide the measurement

Six terms, two of which mean different things to different people — including one that means two things at once.

TENTH-GRADER VERSION

- **Latency** is waiting. **Response time** is waiting plus doing. People use the first word for both, so always say *what* you measured.
- **Utilization** has two meanings: “how much of the time was it busy” and “how much of what it could do is it doing”. Those come apart badly.
- A lift can be moving 100% of the time and still have room for more people. A disk can too.
- **Saturation** is work piling up because it can't be taken yet. Any amount of it is a problem, because it is time someone spent waiting.
- And systems don't get slow gradually — they scale fine, and then there's a bend in the curve where they stop.

THE EVERYDAY VERSION

A lift in a busy building. It is **busy** — moving between floors — 100% of the time, all afternoon. That is time-based utilization, and it is at maximum. But it is not **full**: it can still take more passengers on each trip. That is capacity-based utilization, and it is nowhere near maximum. Now people start waiting in the lobby because every arriving lift is full: that is **saturation**, and the moment it begins, someone's day contains waiting that wasn't there before. Notice that a monitoring dashboard reporting “100% busy” would have looked identical all afternoon, both before and after the queue formed.

The terms, each with its look-alike

TERM	PLAIN DEFINITION	CONCRETE EXAMPLE	BOUNDARY — WHAT IT GETS CONFUSED WITH
Latency	A measure of the time an operation spends waiting to be serviced . In some contexts it is used for the entire operation time.	A page load is DNS latency, then TCP connection latency (the handshake only), then TCP data transfer time — and all three together may be called the latency of something at a higher level.	vs response time , which spans the waiting <i>and</i> the servicing <i>and</i> the transfer of the result. Because the single word is ambiguous, always qualify it: request latency, connection latency, query latency. Naming the target of the measurement is not pedantry; it is the difference between two different numbers.
Why convert to latency	Latency is a time, so different problems can be compared, ranked, and their speedups predicted.	Choosing between 100 network operations and 50 disk operations is a hard question with many factors. Choosing between 100 ms of network I/O and 50 ms of disk I/O is not.	vs IOPS or throughput , which cannot <i>accurately</i> be compared across components or used to calculate a predicted speedup. Where you can convert a metric to time, do — it is what makes prioritisation possible at all.

Utilization (time-based)	U = B/T : the fraction of an observation period during which the resource was busy. Formally defined in queueing theory.	This is what operating system tools report — a disk monitoring tool calls it <i>percent busy</i> , which conveys the underlying measurement better than the word “utilization” does.	vs capacity-based utilization, below. Time-based tells you how busy something is, and near 100% performance can degrade badly under contention — but for a resource that can work on several things at once, it may not degrade at all.
Utilization (capacity-based)	The proportion of the throughput it <i>could</i> deliver that it is currently delivering. The capacity-planning sense of the word.	A disk at 100% capacity utilization cannot accept more work. A disk at 100% time-based utilization only means it was busy the whole interval.	100% busy does not mean 100% capacity. A storage array frequently runs at 100% because some disk is always busy, while plenty of disks sit idle and it can take more work. Ideally you would measure both; usually you cannot, because it would require knowing what the device's controller is doing and predicting its capacity.
Saturation	The degree to which more work is requested of a resource than it can process. Also called pressure .	Begins at 100% <i>capacity-based</i> utilization, and in the idealised picture increases linearly with load beyond it. Measured as a wait-queue length, or as time spent waiting.	vs utilization, which is about busyness. Any degree of saturation is a performance issue , because it is time spent waiting — that is latency by another name. Under the time-based definition, queueing may begin well before 100%, or not at 100% at all, depending on how much parallel work the resource accepts.
Knee point	The boundary where a system's throughput stops scaling linearly with load and contention begins to degrade it.	It may occur when a component reaches 100% utilization — the saturation point — or as it <i>approaches</i> 100% and queueing becomes frequent and significant.	Beyond it, throughput can actually decrease as load increases, because contention and coherency overheads mean less real work gets done. The same curve appears if you scale a resource instead of the load.
Load vs architecture	Two distinct causes of the same symptom. An architecture problem is how the work is done; a load problem is that there is too much of it.	Architecture: a single-threaded application busy on one CPU while others sit idle; or many threads contending for one lock. Load: an application busy on <i>all</i> CPUs with requests still queueing.	The distinction decides the fix. If analysis shows queueing but no problem with <i>how</i> the work is performed, the issue is load — and in a cloud environment that is the point at which you add instances rather than change code.
Cache hit ratio	$\text{hits} / (\text{hits} + \text{misses})$. Higher is better, and the relationship to performance is strongly non-linear .	The improvement from 98% to 99% is far greater than from 10% to 11%, because the two storage tiers differ so much in speed. The greater the difference, the steeper the curve.	vs the miss rate (misses per second), which is proportional to the penalty and often easier to interpret. A 90% hit ratio at 200 misses/s may be worse than an 80% ratio at 20 misses/s — ten times fewer slow reads. To be certain: $\text{runtime} = (\text{hit rate} \times \text{hit latency}) + (\text{miss rate} \times \text{miss latency})$.
Cache warmth	Cold : empty or full of unwanted data, hit ratio near zero. Warm : useful data, but not a high enough ratio to be hot. Hot : commonly requested data and a high hit ratio — over 99%, for example.	Retention and eviction policies decide what stays: most/least recently used, most/least frequently used, and cheaper approximations of them.	The trap is how long warming takes . A storage appliance with 128 GB of memory cache and 600 GB of flash, fed by disks delivering 2,000 reads/s at 8 KB, warms at only 16 MB/s — over two hours for the memory cache and over ten hours for the flash. A benchmark shorter than that is measuring a cold cache.

Three habits the vocabulary is there to support

- **Tune closest to where the work is done.** Tuning at the application level can eliminate work entirely and improve performance by a large factor — of the order of 20×. Tuning down at the storage device level, after the higher-level code has already been executed, may improve the result by percentages — of the order of 20%. But note the separate point: the most effective level to *tune* is not always the most effective level from which to **observe**. A slow query may be best understood from its time on-CPU or from the file system and disk I/O it causes, both of which are visible from operating system tools.
- **Treat every tuning recommendation as perishable.** Performance characteristics change with users, hardware, and software versions — a system limited by its network today may be limited by disk or CPU after an upgrade. A tunable value found on the internet may be right, may have been right once, or may be a temporary workaround for a bug since fixed. Browsing such advice is still useful for discovering *which* parameters exist and have needed changing; your job is then to decide whether they apply here. Keep changed tunables in version control with the reasons.
- **Know why you are stopping.** Three defensible reasons: you have explained the **bulk** of the problem — if a 3× slowdown is accounted for by something worth 12% of CPU, you have not; the potential return is now **less than the cost of the analysis**; or there is a **bigger return elsewhere**. All three require you to have quantified what you found.

CONCEPT BOUNDARY — THE THREE STATES OF YOUR OWN KNOWLEDGE

Known-knowns: you know to check the metric, and you know its value.

Known-unknowns: you know the metric or subsystem exists, and you have not looked. You know you *could* profile what is making the CPUs busy; you have not.

Unknown-unknowns: you do not know the thing exists, so you are not checking it — you may not know that device interrupts can become heavy CPU consumers.

Why it matters: **the more you learn, the more unknown-unknowns become known-unknowns**, which are things you can now go and check. Writing down what you could not measure is therefore a real finding, not a gap in your notes.

Anchor sketch — the words that decide where you look

Legend: bold = a named node

reversed = the node that matters most

Q = cover the page and answer this

an operation's time, split

RESPONSE TIME = waiting + being serviced + transfer

LATENCY = the waiting part
(and, in many contexts, all of it)

+--> so ALWAYS qualify it: request latency,
connection latency, query latency

+--> and convert other metrics TO time where you
can: 100 network I/O vs 50 disk I/O tells you
nothing; 100 ms vs 50 ms is obvious. only time
can be ranked and turned into a predicted speedup.

UTILIZATION has two definitions, and they disagree
time-based $U = B/T$. what fraction of the
interval it was busy. this is what
your tools report.

capacity-based what fraction of what it COULD
deliver it is delivering.

+--> 100% BUSY IS NOT 100% CAPACITY. the lift runs
every minute of the afternoon and still has
room for passengers. so does a storage array
with idle disks.

SATURATION work requested that cannot be serviced yet
begins at 100% CAPACITY utilization

+--> ANY saturation is a performance issue, because
it is queueing, and queueing is latency

KNEE POINT where linear scaling becomes contention
beyond it, more load can buy LESS work, because
contention and coherency overheads consume the gain

Q State both definitions of utilization, and give the example that separates them.

Q Why is latency the metric worth converting other metrics into?

Q Where does saturation begin, and why is any amount of it an issue?

Trade-offs

DECISION	BUYS YOU	COSTS YOU	CHOOSE IT WHEN
Reporting time-based utilization	A number that is actually available from ordinary tools, and a real warning of contention as it approaches 100%.	It says nothing about remaining capacity. Resources that work in parallel can sit at 100% and still accept more.	Always — but read it as “percent busy”, and check saturation to find out what it means.
Tuning at the application level	The largest wins, because work can be eliminated rather than performed faster — of the order of 20×.	It is where change is most frequent, so the target moves; and it is often not the best place to <i>observe</i> from.	First, wherever you have the access and the ability to change it.
Tuning at the device level	Improvements that apply to everything above.	A tax has already been paid executing the higher-level stack, so gains are percentages rather than factors.	After the application, or when the application is not yours to change.

A larger cache	A higher hit ratio, with strongly non-linear returns near the top of the range.	Memory, and a much longer warm-up — long enough that short benchmarks measure the wrong thing.	When the miss penalty is large and the working set fits. Check the miss <i>rate</i> , not just the ratio.
Adopting an internet tunable	Occasionally a genuine quick win, and always a hint about which parameters exist.	It may never have applied to you, may have expired, or may be a workaround for a bug now fixed.	As a source of questions, not answers. Record what you changed and why.
Continuing the analysis	More causes found — real issues often have several.	Engineering time, which has a return of its own elsewhere.	Until you have explained the bulk of the problem, or the return drops below the cost, or a bigger return appears elsewhere.

Say it so you understand it → say it like an expert

SAY IT THIS WAY TO <i>UNDERSTAND</i> IT	SAY IT THIS WAY TO SOUND LIKE AN <i>EXPERT</i>
“The latency is 40 ms.”	“Which latency — DNS, connection, or the whole request? The word covers the wait, and sometimes the whole operation, so it needs a qualifier.”
“The disk is at 100%, that’s our bottleneck.”	“100% busy, or 100% capacity? Busy means it never idled in the interval. Check saturation — a queue is what tells us it can’t take more.”
“It’s only 60% utilized, we’re fine.”	“Over what interval? A five-minute average hides seconds at 100%, and a short burst is enough to produce queueing.”
“It gets slower as we add load.”	“Where’s the knee point? Below it we’re scaling linearly; above it, contention means added load can buy us less work, not more.”
“The cache hit ratio is 90%, that’s good.”	“What’s the miss rate? 90% at 200 misses a second can be worse than 80% at 20 — the penalty is what costs us, not the ratio.”
“We found the cause.”	“How much of the problem does it explain? If we’re 3× slow and this accounts for 12%, we’ve found <i>an</i> issue, not <i>the</i> issue.”

PART 1 · QUESTION 1

A team reports that their storage tier is “maxed out”: the monitoring dashboard shows the array at 100% utilization all through the working day. They have quoted for a second array. Separately, users complain of slow requests only between 09:15 and 09:40 — a window in which the dashboard shows nothing unusual, because it plots five-minute averages. The array's own queue-length metric is not collected.

(a) Explain why 100% utilization on the dashboard may not mean the array is out of capacity, and give the everyday example that makes the distinction. **(b)** Explain how the morning window can be a real problem that the dashboard cannot show, and name the metric that would settle it. **(c)** Say what you would collect before approving the purchase, and classify the missing queue-length metric in terms of what you know about your own knowledge.

[illegible]

PART 1 · QUESTION 2

A service resizes images. Someone proposes adding an in-memory result cache and runs a ten-minute benchmark, which shows a 4% improvement; they conclude caching is not worth it. A colleague proposes instead a configuration change deep in the storage layer, found on a forum post, which they say gave someone else “a 3× speedup”. A third engineer notes that the service's own code re-reads and re-decodes the same source image several times per request.

(a) Give two independent reasons the benchmark result is not evidence against the cache. **(b)** Assess the forum tunable, naming what it is and is not useful for. **(c)** Rank the three proposals and justify the ranking with the general principle about where tuning pays — including the caveat about where to *observe* from.

2×2 — busy, and able to take more

COLUMNS: CAN IT ACCEPT MORE WORK RIGHT NOW? →

	Yes	No
BUSY — time-based utilization near 100%	100% busy, not 100% capacity The lift moving every minute with room in it. The storage array where some disk is always busy and many are idle. A disk buffering writes to complete later. <i>Move:</i> do not treat this as a bottleneck on the strength of the utilization figure alone. It is the cell that makes “100%” a bad alarm, and it is why you check saturation before concluding anything.	Saturation begins here 100% capacity-based utilization: work arrives that cannot be serviced and starts to queue. <i>Move:</i> this is the real bottleneck, and the metric that proves it is a wait queue, not a percentage. Any non-zero saturation is a problem, because queueing is latency. Beyond this point, saturation keeps growing with load — roughly linearly, in the idealised picture — while utilization cannot report anything new.
NOT BUSY	Idle, and fine Low utilization, no saturation, no errors. <i>Move:</i> nothing — and note that this is more useful than it sounds. Ruling a resource out narrows the investigation, and that process of elimination is most of what a systematic check buys you.	Blocked by something other than work A resource that is not busy and still cannot take more: a limit imposed from outside, or a component in a degraded or failing state. <i>Move:</i> look for a resource control — a per-tenant cap can produce allocation failures or throttling while the host is calm — or for errors, which are worth checking first precisely because they are objective and quick to rule out.

Read it as: the row is what your tools tell you and the column is what you actually need to know, and the top row is where they diverge. Both top cells report identical utilization; only one of them is a problem. The metric that separates them is saturation.

CARRY-OUT RULE FROM THIS PART

Say which definition you are using, then find the **queue**. Utilization near 100% is a prompt to look, not a conclusion — the resource may have plenty of headroom. Saturation is the metric that decides, and unlike utilization it has no ambiguous form: any amount of it is time somebody spent waiting.

Where you stand decides what you see

Two perspectives, three anti-methods with one shared tell, and the procedures that replace them.

TENTH-GRADER VERSION

- You can investigate a slow system by looking at its *parts* — processors, memory, disks — or by looking at the *work* being asked of it.
- Most documentation teaches the first one, so most people don't realise the second exists.
- The bad ways of investigating all share one feature: someone has a theory and no evidence for it.
- The cheapest good move is to just *ask six questions* before touching anything. Problems get solved this way without anyone logging in.
- After that, work like a scientist: guess, predict what the guess implies, test it, and let the data kill bad guesses fast.

THE EVERYDAY VERSION

A doctor with a patient who feels unwell. One approach is to work through the body's systems in turn — heart, lungs, bloods — which is thorough and belongs to whoever is responsible for the machinery. The other is to start from the complaint: when does it hurt, what were you doing, what changed. Both are legitimate and they find different things. What is *not* legitimate is the version where the doctor announces “it's probably your diet” before asking a single question and refers you to a dietician — and, when that comes back clear, announces it is probably your sleep. That is not a diagnosis; it is a hypothesis with nothing behind it, and it costs other people's time.

The terms, each with its look-alike

TERM	PLAIN DEFINITION	CONCRETE EXAMPLE	BOUNDARY — WHAT IT GETS CONFUSED WITH
Resource analysis	Bottom-up. Begin with the system resources: processors, memory, disks, network interfaces, buses, interconnects.	Used for investigating whether a type of resource is responsible, and for capacity planning. Best-suited metrics: IOPS, throughput, utilization, saturation.	vs the assumption that it is <i>the</i> perspective. It is common partly because the widely available documentation teaches it — the operating system “stat” tools. Reading that documentation, remember it is one perspective, not the only one. It is typically performed by whoever is responsible for the physical resources.
Workload analysis	Top-down. Examine the load applied and how the application is responding.	Three targets: requests (the load applied), latency (the response time), and completion (looking for errors). Best-suited metrics: throughput and latency.	vs resource analysis, which starts at the other end of the stack. Note the third target: a request may complete <i>quickly</i> and with an error status that causes a retry, accumulating latency invisibly. And the fastest query is still the one you don't do at all — examining request attributes can identify work to eliminate even when latency is fine.

Streetlight anti-method	The absence of a deliberate method: choose the observability tools you are familiar with, or find at random, and see if anything obvious shows up.	Named for the drunk searching for keys under the streetlight — not because he lost them there, but because that is where the light is best.	vs the tools method, which is the systematic version of the same limitation. The specific danger here is not that it finds nothing; it is that what it finds may be an issue but not the issue , and nothing in the approach quantifies the finding well enough to tell.
Random change anti-method	Experimental guessing: pick something to change, change it one way, measure; change it the other way, measure; keep whichever beat the baseline; repeat.	It can eventually find tuning that works for the tested workload.	vs a controlled experiment. It is very time-consuming, and it leaves behind tuning nobody understood in the first place — possibly working around a bug that is later fixed, after which the tuning stays and no longer makes sense. The other risk is a poorly understood change causing a worse problem at peak load.
Blame-someone-else anti-method	Find a component you are not responsible for, hypothesise that it is the cause, redirect the issue to that team, and when proven wrong, repeat.	“Maybe it's the network — can someone check with the network team?”	The identifying feature, and the one worth memorising, is a lack of data leading to the hypothesis . The countermeasure is procedural: ask the accuser for the screenshots showing which tools were run and how the output was interpreted, and take those to someone else for a second opinion.
Ad hoc checklist method	Stepping through a canned list of checks, each with an interpretation and a fix. Common in support, often under time pressure.	An entry looks like: run this tool, read this column; if the value is consistently over this threshold under load, then the disk is slow or overloaded.	Not an anti-method exactly: it delivers the most value in the shortest time, and it is an effective way to make sure a whole team can check for common issues. But every entry is a point-in-time recommendation that decays, and checklists skew toward issues with easily documented fixes — tunable settings — rather than custom fixes to code or environment.
Problem statement	Six questions asked before anything else, which is why it should be the first method you attempt on a new issue.	What makes you think there is a problem? Has it ever performed well? What changed recently — software, hardware, load? Can it be expressed as latency or runtime? Does it affect others? What is the environment, with versions and configuration?	vs starting with tools. Asking and answering these often points straight at the cause: performance issues have been solved this way alone, over the phone, without logging in to a server or looking at a single metric.
Scientific method	Question, hypothesis, prediction, test, analysis. The question is the problem statement; the test may be observational or experimental.	Observational: hypothesise noisy neighbours are contending for disk I/O; predict file system latency will dominate query latency; trace it and find it is under 5%; conclude the file system and disks are not responsible.	Note what that example achieves: the issue is unsolved, and large parts of the environment have been ruled out . Also note the discipline for experimental tests — if the change doesn't fix it, reverse it before forming a new hypothesis , because changing several things at once makes it impossible to tell which mattered.
Negative test	Deliberately making performance <i>worse</i> to learn about the system.	Suspecting that managing a larger cache costs compute, progressively shrink the record size so more records are needed for the same data, and predict performance degrades progressively.	vs an experiment that tries to help. Often the easier experiment: artificially reducing a cache is usually cheaper than adding memory, and a confirmed prediction is evidence either way.

Diagnosis cycle	hypothesis → instrumentation → data → hypothesis. A loop rather than a line.	Like a doctor running a series of small tests, refining the hypothesis after each.	vs the scientific method, which it closely resembles. The emphasis differs: the cycle stresses that data leads quickly to the next hypothesis. Both have a good balance of theory and data, and the operative advice is the same — get to data fast, so bad theories are identified early and discarded.
Tools method	List the tools available, list the useful metrics each provides, list how to interpret each metric. The result is a prescriptive checklist.	Fairly effective in practice: it does identify certain resource bottlenecks and errors, though not always efficiently.	vs a method that starts from questions. It relies exclusively on the tools you have or know , which gives an incomplete view — and, worse, the user is unaware the view is incomplete and may remain so. Issues needing custom tooling may never be found. It also scales badly: many tools with overlapping functionality cost time to evaluate.

CONCEPT BOUNDARY — THE ANTI-METHODS ARE NOT STUPID, THEY ARE UNTETHERED

What the three anti-methods have in common is not laziness. It is that the hypothesis arrives *before* the data, or without any data at all — which is stated explicitly as the identifying feature of blame-someone-else and is equally true of the other two. The ad hoc checklist is deliberately not among them: it is a real method, and its weakness is decay rather than untetheredness.

What separates the good methods is not sophistication either. The problem statement is six questions and no tools; the diagnosis cycle is four words. What they share is that **data comes first and the finding gets quantified**, which is what lets you rule things out, rank what you found, and know when to stop.

The test to apply to yourself: what did I observe before I formed this theory, and how big is the thing I think I have found?

Anchor sketch — two perspectives, four traps, three procedures

Legend: bold = a named node **reversed = focus node** Q = self-test cue

two perspectives on the same stack

RESOURCE ANALYSIS bottom-up. CPUs, memory, disks, interfaces, buses. utilization, saturation, IOPS, throughput.

WORKLOAD ANALYSIS top-down. requests, latency, completion (errors). throughput and latency.

+--> neither is "the" perspective. the widely available documentation teaches the first, which is why people think it is the only one.

|

the three ANTI-METHODS, and the tell they share

STREETLIGHT you run the tools you know, because that is where the light is. it may find an issue -- but not necessarily THE issue.

RANDOM CHANGE change, measure, keep what helped. leaves tuning nobody understood, sometimes working around a bug that later gets fixed.

BLAME SOMEONE ELSE redirect to a team you are not on. the tell: A HYPOTHESIS WITH NO DATA BEHIND IT. ask for the screenshots.

+--> all three: the theory arrives before the data (the AD HOC CHECKLIST is NOT one of them: it is a real method, fast and genuinely valuable -- but a point-in-time recommendation that decays, and skewed to easily documented fixes.)

|

what to do instead, in order

- 1 **PROBLEM STATEMENT** six questions, asked FIRST. issues HAVE been solved this way without logging in to anything.
- 2 **SCIENTIFIC METHOD** question . hypothesis . prediction . test . analysis. ruling things OUT is a result. reverse a failed change before forming the next hypothesis.
- 3 **DIAGNOSIS CYCLE** hypothesis -> instrumentation -> data -> hypothesis. get to data fast, so bad theories die early.

Q Name the two perspectives, who typically uses each, and the metrics each favours.

Q Name the three anti-methods, and the single feature that identifies all of them.

Q Give the six problem-statement questions, and say why this method comes first.

Trade-offs

DECISION	BUYS YOU	COSTS YOU	CHOOSE IT WHEN
Starting from resources	Systematic coverage of the machinery, and the metrics for capacity planning.	It says little about whether the <i>work</i> being asked is reasonable, or where in a request the time went.	You own the infrastructure, or you suspect a resource limit.
Starting from the workload	Latency attributed to the request, errors that cause silent retries, and work you could eliminate entirely.	It can be blind to a resource limit that is degrading everything at once.	You own the application, or the complaint is expressed as user-visible slowness.

An ad hoc checklist	The most value in the shortest time, and a whole team able to check for common issues.	It decays, and it steers toward issues with documentable fixes rather than the ones needing custom work.	Short engagements and support rotations — with a standing commitment to refresh it.
The tools method	A prescriptive list: which tool, which metric, how to read it. Genuinely finds bottlenecks and errors.	Bounded by the tools you have, and the boundary is invisible from inside — you don't know what you can't see.	As a supplement, after a method that starts from questions has told you what you are unable to answer.
The problem statement first	The cheapest possible method: no tools, no access, and it frequently identifies the cause outright.	Nothing, other than needing someone to answer honestly.	Always, on every new issue, before anything else.
An experimental test	A direct answer — the change either fixed it or did not.	Risk in production, and the discipline of reversing failed changes before trying the next one.	When the observational route is unavailable or slower, and the change is safe to make and unmake.

Say it so you understand it → say it like an expert

SAY IT THIS WAY TO <i>UNDERSTAND</i> IT	SAY IT THIS WAY TO SOUND LIKE AN <i>EXPERT</i>
“I checked the usual tools and nothing looked odd.”	“That's the streetlight method — bounded by what we happen to know. Let's enumerate the questions first and then find tools for them.”
“It's probably the network.”	“What data led to that? A hypothesis with nothing behind it is how we burn another team's week. What did you run, and what did the output say?”
“We changed the setting and it got faster.”	“Do we know why? Tuning nobody understands survives past the bug it was working around, and then nobody dares remove it.”
“Let's start looking at the servers.”	“First: what makes them think there's a problem, has it ever been good, what changed, can we express it as latency, who else is affected, what's the environment?”
“We ruled out the disks — dead end.”	“That's a result, not a dead end. We've eliminated a large part of the environment; next hypothesis.”
“We tried three things and it's better now.”	“Then we don't know which one mattered. Reverse a failed change before making the next one, or we can't attribute anything.”
“Our runbook covers this.”	“Good — and when was it last refreshed? Every entry is a point-in-time recommendation, and it'll be biased toward the fixes that were easy to write down.”

PART 2 · QUESTION 1

An engineer is paged for slow checkout. Within ten minutes they have opened their usual dashboard, seen elevated disk activity, raised a ticket with the storage team, and gone back to other work. The storage team spends two days demonstrating the array is healthy. Meanwhile a second engineer changes three configuration values at once during the incident; performance improves and the changes are left in place. Nobody has yet asked the checkout team anything.

(a) Name the methods being used by each engineer and state the feature they share. **(b)** Say what should have happened first, list what it would have established, and explain why it costs almost nothing. **(c)** Name two distinct problems created by the second engineer's approach, one of which will only appear much later.

PART 2 · QUESTION 2

A platform team maintains a 30-entry checklist, built over two years, that every engineer runs on any performance complaint. It is fast and frequently effective. This quarter it has failed three times: twice it reported nothing while users were clearly affected, and once it recommended a tunable change that made things worse. The team's proposal is to add more entries and to buy two more observability tools.

(a) Say what the checklist is good for and name its two structural limitations. **(b)** Explain why adding tools addresses the wrong constraint, and name the specific blind spot the tools-oriented approach has. **(c)** Give the sequence you would put in front of the checklist, and say what should be recorded when a question turns out to be unanswerable.

	From a tool you have	From a question you need answered
NOTHING was observed first	Streetlight You open what you know and look for something odd. Sometimes it works. <i>Move:</i> notice that a finding here is unranked — you cannot tell an issue from <i>the</i> issue, because nothing in the approach quantifies what it found against the size of the problem.	Random change · blame someone else A hypothesis with nothing behind it: change things until the symptom moves, or hand it to a team you are not on. <i>Move:</i> stop. Ask what was observed before the theory. For the second, ask for the screenshots and the interpretation — and note that the cost lands on other people, not on you.
DATA came first	Tools method · ad hoc checklist Systematic and genuinely effective: list your tools, their metrics, and the interpretations. <i>Move:</i> use it, and hold the limitation in mind — the view is bounded by the tools you have, and <i>you cannot see the boundary from inside</i>. The checklist has the same shape plus an expiry date.	Problem statement, then a cycle Six questions first. Then question, hypothesis, prediction, test, analysis — or the tighter loop of hypothesis to instrumentation to data and back. <i>Move:</i> start here on every issue. Enumerate the questions before looking for tools, so that a question you cannot answer becomes a recorded known-unknown rather than an invisible gap. Ruling things out counts as progress.

Read it as: the row is the one that separates method from anti-method — in every cell of the top row the step was taken with nothing observed behind it, whether that is a theory with no evidence or no theory at all. The column decides how complete your view can be: a tool-first approach is bounded by your toolbox, and the danger is that the boundary is invisible from where you are standing.

CARRY-OUT RULE FROM THIS PART

Ask what you observed before you formed the theory. If the honest answer is “nothing”, you are in an anti-method regardless of how reasonable the theory sounds — and the cheapest correction is six questions that need no tools and no access at all.

The methods worth running

A small number of procedures that answer “where do I start” for different shapes of problem.

TENTH-GRADER VERSION

- For every *part* of the machine, ask three things: how busy, how much waiting, how many errors. That's one method and it covers a lot.
- For every *service*, ask three different things: how many requests, how many failed, how long they took.
- To find work that shouldn't exist at all, look at what's being asked of the system rather than at the system.
- To find where the time goes, split the time in two, follow the bigger half, and repeat.
- And check the boring stuff: is anything just misconfigured? That check takes minutes and nobody remembers to do it.

THE EVERYDAY VERSION

A restaurant kitchen where the food is coming out late. One approach walks the stations — grill, fryer, pass — and asks of each: how busy is it, is there a queue building, is anything broken? That is a complete sweep of the machinery and it takes minutes. Another approach stands at the pass and asks of the orders: how many are coming in, how many are being sent back, how long are they taking? That is the diners' view, and it tells you whether the problem is that the kitchen got slower or that more people walked in. And before either, there is the check nobody does: with the kitchen empty, is the grill actually on the right setting, and is one of the two fryers still switched off from last night?

The terms, each with its look-alike

TERM	PLAIN DEFINITION	CONCRETE EXAMPLE	BOUNDARY — WHAT IT GETS CONFUSED WITH
The USE method	For every resource , check utilization, saturation and errors . Used early in an investigation to find systemic bottlenecks.	Utilization as a percentage over an interval; saturation as a wait-queue length; errors as a count. Roughly 30 combinations for a typical server.	vs the tools method, and this is the key contrast: USE iterates over resources, not tools . You build the complete list of questions first and <i>then</i> look for tools — so a question with no available tool becomes a recorded known-unknown, which is itself useful, rather than an invisible gap.
The USE order	Errors first, saturation second, utilization third.	Errors are usually objective rather than subjective, so they are quick to interpret and time-efficient to rule out. Saturation is next because <i>any</i> level of it can be an issue.	vs starting with utilization, which is the metric that most needs interpretation — and the one whose two definitions can mislead. Note also that errors matter even when recoverable: operations that fail and retry, or a failed device in a redundant pool, degrade performance without being noticed.

USE on software and limits	The same three questions applied to software resources and to imposed caps.	A lock: utilization is time held, saturation is threads queued on it. A thread pool: utilization is time busy, saturation is requests waiting. Process, thread and file-descriptor capacity: usage, waiting on allocation, and allocation failures.	Also applies to resource controls : a tenant's memory usage against its cap is capacity utilization, and limit-imposed allocation errors or swapping are saturation — even when the host system is not under memory pressure . And for a service architecture, utilization can be average CPU across the cluster, saturation approximated as the gap between the 99th percentile latency and the average — on the assumption that the 99th is saturation-driven — and errors as request errors.
The RED method	For every service , check the request rate, errors and duration . Aimed at services, from the user's point of view.	Draw the service architecture and make sure all three are monitored for each service. For duration, consider distribution statistics as well as the average.	vs USE, and the two are complementary rather than competing: USE for machine health, RED for user health . RED's distinctive contribution is the request rate, which separates load from architecture: duration up with rate flat points at the service itself; both up means the problem may be one of load .
Workload characterization	Four questions about the input rather than the resulting performance: who is causing the load, why, what are its characteristics, and how is it changing over time.	Who: process, user, remote address. Why: code path or stack. What: rate, throughput, direction, type — with variance where appropriate. How: is there a daily pattern?	vs analysis of the system, which may find nothing wrong: the architecture can be sound and the load simply more than it can handle. Check all four even when you are confident of the answers — the worked example is discovering that the clients are not the expected web servers but the entire internet, i.e. an attack. If the work cannot be eliminated, throttling it with resource controls is the fallback.
Drill-down analysis	Three stages: monitoring (high-level statistics over time, and alerting), identification (narrow to particular resources or areas), analysis (root-cause and quantify).	Monitoring is environment-wide and reveals long-term patterns that short command-line sessions miss. Analysis is where tracing and profiling tools come in, and where custom tooling gets written.	Pair it with Five Whys : ask why, answer, and repeat about five times. The worked example runs from “queries are slow” through paging, memory growth and allocator behaviour to a memory fragmentation issue — a real case that ended in a fix to a system library. Persistent questioning is the mechanism; stopping at the first plausible answer is the failure.
Latency analysis	Take the total time, split it into components, and keep subdividing the largest component until the cause is found and quantified.	Is there a latency issue? On-CPU or off-CPU? Waiting on what — the file system? Disk I/O or lock contention? Queueing or servicing? Initialisation or data transfer?	Each step divides the latency into two parts and analyses the larger one — a binary search of latency . Similar to drill-down in that it descends the stack, but the organising principle is different: drill-down narrows by <i>area</i> , latency analysis narrows by <i>where the time is</i> .
Event tracing	Inspecting individual events rather than summaries: system calls, I/O, packets, queries.	For each event look for the input (type, direction, size), the times (start, end, latency) and the result (error status, transfer size).	vs summaries such as operations per second or average latency, where important detail is lost. The subtle payoff: a latency outlier may be caused by previous events rather than its own properties — the event at the tail of a queue is slow because of what was queued ahead of it, and only the traced sequence shows that.

Baseline statistics	Collecting the full set of metrics while the system is under “normal” load, and recording them for later comparison.	Collected at regular intervals and, importantly, before and after changes . It can be a script that runs the observability tools and captures their output.	vs a monitoring dashboard, which covers only the metrics someone thought to monitor. Baselines answer “is this number normal for this server?” for the many command-line details that are not monitored. If no baseline exists, summary-since-boot figures from kernel counters are coarse but better than nothing. Beware the overhead of tracing tools in a baseline, so as not to perturb production.
Static performance tuning	Checking the configured architecture, with the system at rest and no load applied . Four questions per component: does it make sense, does its configuration suit the intended workload, was it auto-configured well, and has it degraded after an error?	An interface negotiated to a tenth of its speed; a failed disk in a redundant pool; an old version; a nearly-full file system; a record size mismatched to the workload; a debug mode left on; a server accidentally routing; a service reaching across data centres for something available locally.	vs every other method here, which studies the dynamic performance of applied load. This is the one performed without load at all. These issues are easy to check for; the hard part is remembering to do it .
Micro-benchmarking	Testing simple, artificial workloads, measuring the time for a large number of operations and calculating the average.	System call time; file system reads from a cached file across sizes from one byte to a megabyte; network throughput between endpoints across socket buffer sizes. Note the second dimension in each — vary something.	vs macro-benchmarking, which simulates a real-world workload and is complex to conduct and to understand. With fewer factors in play, micro-benchmarks are easier to reason about — and a network throughput test in particular can identify external network bottlenecks that would otherwise be difficult to spot, by examining the transport counters during a production workload. You may use a benchmark tool, or a load generator plus separate observability; either is fine, and cross-checking with a second tool is safest.

Cache tuning, in order

1. **Aim to cache as high in the stack as possible**, closest to where the work is performed. This reduces the operational overhead of each hit, and that level usually has more metadata available to inform the retention policy.
2. Check the cache is **enabled and working** at all.
3. Check the **hit/miss ratio and the miss rate**.
4. If the size is dynamic, check its **current size**.
5. **Tune the cache for the workload**, using whatever tunables exist.
6. **Tune the workload for the cache** — including removing unnecessary consumers, which frees space for the traffic you care about.

Two things to watch. **Double caching**: two caches both consuming main memory to hold the same data twice. And the **size of the win at each level**: improving a first-level processor cache may save nanoseconds, because a miss is served by the next level down; improving a last-level cache may avoid a far slower memory access and deliver a much larger overall gain.

CONCEPT BOUNDARY — USE IS NOT A CHECKLIST OF TOOLS, AND FINDING SOMETHING IS NOT FINISHING

USE iterates over resources and only then asks what can measure them. The tools method iterates over tools. That difference is what converts an unanswerable question into a known-unknown instead of a blind spot.

And a system may have more than one problem. The first thing USE finds may be a problem but not *the* problem — each discovery should be investigated with other methods, and then you return to USE and continue iterating over the remaining resources.

Also worth stating: the negative result is valuable. Low utilization, no saturation, no errors, across the board, tells you it is probably not a resource problem — and narrowing the scope that fast is most of what the method buys you.

Anchor sketch — the methods, and what each one is the answer to

Legend: **bold** = a named node **reversed** = focus node **Q** = self-test cue

USE for every RESOURCE: utilization, saturation, errors
order: ERRORS first -- objective, quick to rule out
SATURATION next -- any amount is an issue
UTILIZATION last -- needs the most interpreting
iterate over RESOURCES, not tools: build the list of
questions, THEN go looking. a question with no tool is
a recorded KNOWN-UNKNOWN, not a blind spot.
~30 combinations. works on software too: locks, thread
pools, fd caps, and per-tenant resource limits.

RED for every SERVICE: request rate, errors, duration
USE = machine health. RED = user health.
the RATE is the clue: duration up + rate flat
-> architecture. both up -> load.

WORKLOAD CHARACTERIZATION who . why . what . how changing
aimed at the INPUT, not the result. check all four even
when you are sure -- that is how you find the load that
should not exist at all.

DRILL-DOWN monitor -> identify -> analyse, + FIVE WHYS

LATENCY ANALYSIS split the time in two, follow the larger
half, repeat. a binary search of latency.

EVENT TRACING input . times . result, per event.
an outlier may be caused by the events BEFORE it.

and the ones people skip

STATIC PERFORMANCE TUNING check the configuration with
NO load at all. easy to do, easy to forget.

BASELINE STATISTICS so "is this normal?" has an answer
later. take one before every change.

MICRO-BENCHMARKING simple artificial workload, fewer
factors, easier to understand.

Q Give USE's three metrics, the order to check them, and the reason for that order.

Q How does the request rate in RED separate a load problem from an architecture problem?

Q Which method runs with no load applied, and why is it the one people forget?

THE PERFORMANCE MANTRAS, MOST EFFECTIVE FIRST

1. Don't do it — eliminate unnecessary work. **2. Do it, but don't do it again** — caching. **3. Do it less** — reduce the frequency of refreshes, polling, updates. **4. Do it later** — write-back caching. **5. Do it when they're not looking** — schedule for off-peak hours. **6. Do it concurrently** — single-threaded to multi-threaded. **7. Do it more cheaply** — buy faster hardware.

The ordering is the content. Buying hardware is last not because it never works but because six cheaper things outrank it, and the first — not doing the work at all — is where the best wins in this entire booklet come from.

Trade-offs

DECISION	BUYS YOU	COSTS YOU	CHOOSE IT WHEN
The USE method	Every resource checked quickly against a small number of key metrics, plus a list of what you <i>cannot</i> measure.	Around 30 metric combinations to assemble, some hard to obtain and some impossible. It suits resources that degrade under load, not caches, which improve.	Early in any investigation, to find systemic bottlenecks and to eliminate resources fast.
The RED method	User-visible health per service, and an immediate read on load versus architecture from the request rate.	It says nothing about which resource is responsible; it is the complement to USE, not a replacement.	Service architectures, and any time the complaint is user-facing.
Workload characterization	The chance to eliminate work rather than speed it up — the largest wins available.	Effort spent on the input rather than the system, which finds nothing if the load is genuinely reasonable.	When the architecture looks sound, or before designing a simulation benchmark.
Event tracing	Detail that summaries destroy, including outliers and the events that caused them.	Volume, and the overhead of collection — which can perturb the thing you are measuring.	When a summary looks fine and users disagree, or when you need per-event attribution.
Baseline statistics	An answer to “is this normal?” for metrics no dashboard covers, and before/after evidence for changes.	Storage, discipline, and care with tracing overhead so the baseline does not disturb production.	Routinely, and always immediately before and after a change.
Micro-benchmarking	Few factors, easy to conduct and to interpret; a network throughput test in particular can identify external network bottlenecks that are otherwise hard to see.	An artificial workload, which may not represent production at all.	To isolate one component's behaviour, or to test a system with no production load yet.

Say it so you understand it → say it like an expert

SAY IT THIS WAY TO UNDERSTAND IT	SAY IT THIS WAY TO SOUND LIKE AN EXPERT
“Let's check everything.”	“USE: every resource, utilization, saturation, errors — errors first because they're objective, saturation next because any amount matters.”
“We don't have a metric for that.”	“Then write it down as a known-unknown. Enumerating what we can't measure is a result — it's the gap the tools method would have hidden from us.”

“The service got slower.”	“Did the request rate change? Duration up with rate flat is the service itself; both up is load, and that's a different fix.”
“We need to make the queries faster.”	“Or not run them. Characterise the workload first — who, why, what and how it's changing. The best wins come from eliminating work.”
“That one request took 900 ms.”	“Look at what preceded it. An outlier at the tail of a queue is slow because of the events ahead of it, not its own properties.”
“Is 4,000 context switches a second high?”	“Compared with what? Without a baseline we can't say. Failing that, summary-since-boot is coarse but better than nothing.”
“We've found the cause.”	“Found <i>a</i> cause. Systems often have several — investigate this one, then go back and finish iterating the resources.”

PART 3 · QUESTION 1

A payments service has slowed down over three weeks. Latency at the 99th percentile has tripled; the request rate is unchanged. An engineer has spent two days profiling application code and found nothing conclusive. The service runs in a container with a memory cap; the host is comfortably provisioned and its dashboards look calm. Nobody has looked at the service since it was deployed eleven months ago.

(a) Using the rate-and-duration relationship, say what class of problem this is and what it rules out. **(b)** Name the method you would run first, give its three metrics in the correct order with the reason for that order, and say specifically how you would apply it to a capped container — including why the calm host is not reassuring. **(c)** Name one further method that is cheap, requires no load, and is very likely to be relevant given the last sentence of the scenario.

PART 3 · QUESTION 2

A reporting database is slow every weekday morning. Average query latency looks acceptable, but users describe some queries as “hanging”. The team has confirmed the hardware is healthy and the query plans are unchanged since last year, and is now proposing to double the number of database instances. No one has examined which clients are connecting or what they are asking for.

(a) Name the method that addresses the gap in the last sentence, give its four questions, and say why they should be asked even when the team is confident of the answers. **(b)** The average latency is acceptable and some queries hang. Name the method that resolves that contradiction and explain what it looks at that a summary cannot. **(c)** Before doubling the instances, name what must be established, and give the ordering principle that says where to look for a fix first.

2x2 — what you are looking at, and whether the load is real

COLUMNS: IS REAL LOAD BEING APPLIED WHILE YOU LOOK? →

	Yes — production load, dynamic performance	No — at rest, or load you supplied yourself
You are looking at RESOURCES	The USE method Every resource: utilization, saturation, errors — errors first, saturation second, utilization last. <i>Move:</i> run it early. It finds systemic bottlenecks fast, and its negative result is worth as much as its positive one: low utilization, no saturation and no errors across the board eliminates a whole class of cause in minutes.	Static performance tuning With no load at all, ask of each component: does it make sense, does the configuration suit the workload, was it auto-configured well, has it degraded after an error? <i>Move:</i> do it, because almost nobody does. A negotiated-down interface, a failed disk in a redundant pool, a debug mode left on, a nearly-full file system. These are easy to check for; the hard part is remembering to — which is precisely why this cell is the focus.
You are looking at REQUESTS	RED, and the workload methods Every service: rate, errors, duration. Then characterise the workload — who, why, what, how changing — and split the latency in half until it confesses. <i>Move:</i> read the rate first, because it separates load from architecture before you have done any real work. Then look for load that should not exist at all.	Micro-benchmarking You supply an artificial workload to a system that has no real one, vary one dimension, and measure. <i>Move:</i> use it to isolate a single component, or to size something that has no production traffic yet. Note the difference from the cell above it: static tuning applies <i>no</i> load, whereas here you apply an artificial one. Remember what you have bought — fewer factors and therefore a clearer answer to a question that may not be the one production is asking.

Read it as: the left column is where all the famous methods live and where all the attention goes. The right column is the one you reach for when production traffic is not available to you — or, in the focus cell, when you deliberately take it away. A configuration fault found at rest would have survived every dynamic method in the left column, quietly, for as long as nobody looked.

CARRY-OUT RULE FROM THIS PART

Pick the method from the shape of the question, not from the tool you have open. Resource or request; under load or at rest. And whichever you run, quantify what it finds — a system usually has more than one problem, and without a size you cannot tell whether you have found the one that matters.

The numbers will lie to you

Scalability models, queueing theory, and the statistics that describe nothing that actually happened.

TENTH-GRADER VERSION

- Systems don't slow down gradually. They scale fine, then bend, then get worse — and there are only a few shapes that bend can take.
- Queues have maths. A disk busy much past 60% of the time already answers about twice as slowly as an idle one.
- An average is a single number standing in for a lot of numbers. Sometimes there's nothing in the middle for it to stand for.
- If some requests take under a millisecond and the rest take about 7, the average can come out at 3.3 — and nothing that happened took 3.3.
- So whenever someone says “the average is fine”, the right question is: what did the whole distribution look like?

THE EVERYDAY VERSION

A river with an average depth of six inches. You can walk it, obviously. Except the average is made of a wide shallow bank and a narrow deep channel, and the man in the proverb drowned in it. Now consider a single-lane toll booth. When it is idle you are through immediately; once it is busy rather more than 60% of the time you wait, on average, twice as long as when it was empty — not 60% longer, twice. And if someone tells you the booths ran at 40% utilization across the whole day, you still cannot say whether anyone queued, because the whole day contains a rush hour in which they were certainly at 100%.

The terms, each with its look-alike

TERM	PLAIN DEFINITION	CONCRETE EXAMPLE	BOUNDARY — WHAT IT GETS CONFUSED WITH
Scalability profiles	Five shapes you can identify visually, without any model, by plotting delivered performance against the thing you are scaling.	Linear ; contention for shared components used serially; coherence , where the tax of keeping data consistent outweighs the gain; a knee point where the profile changes; and a ceiling , a hard limit.	vs a model. Visual identification is easy and effective, and worth doing first — the observed knee point can then be checked against a plausible cause, and confirmed experimentally by repeating the test on a system where that cause would move.
Amdahl's Law of Scalability	$C(N) = N / (1 + \alpha(N - 1))$, where C is relative capacity, N is the scaling dimension, and α is the degree of seriality (between 0 and 1).	Fitted by collecting data across a range of N — by observation or by micro-benchmarking — then performing regression to find α , then plotting the model with the data.	This is the contention profile: it accounts for the serial part of a workload that cannot run in parallel, and α is exactly how far the system deviates from linear.

Universal Scalability Law	$C(N) = N / (1 + \alpha(N - 1) + \beta N(N - 1))$, adding β , the coherency parameter .	This is the coherence profile, and it includes the effects of contention as well.	vs Amdahl, which it contains: when $\beta = 0$ it becomes Amdahl's Law . The practical value of fitting a model at all is that a deviation between model and data is informative — either your understanding of the system is wrong, or the system's real scalability is.
Little's Law	$L = \lambda W$: the average number of requests in a system equals the average arrival rate multiplied by the average time each spends in it.	Applied to a queue: L is the number waiting, W is the average wait.	One of the foundations queueing theory is built on, alongside probability distributions, stochastic processes and Erlang's C formula. Queueing systems let you answer questions like: what will mean response time be if load doubles, or if we add a processor?
Kendall's notation	A/S/m : the arrival process, the service time distribution, and the number of service centres.	M/M/1 Markovian arrivals and service times, one server. M/M/c the same, multi-server. M/G/1 Markovian arrivals, general service times — commonly applied to rotational disks. M/D/1 Markovian arrivals, deterministic (fixed) service times.	An extended form adds buffers, population size and service discipline. In queueing theory the service centres are usually called <i>servers</i> , which is not the same word you use for a machine.
M/D/1 past 60%	$r = s(2 - \rho) / 2(1 - \rho)$, where r is response time, s is service time and ρ is utilization.	Past 60% utilization the mean response time has doubled . By 80% it has tripled . An idle rotational disk might serve I/O in about 1 ms; under load that can approach 10 ms.	Why disk utilization is a problem well before 100%: it is a queueing system whose requests typically cannot be interrupted and must wait their turn. CPUs differ — higher-priority work can preempt. And this model is the <i>best</i> case: variation in service time drives the mean higher, and the 90th and 99th percentiles degrade much faster than the mean beyond 60%.
Resource limit analysis	Measure the request rate, measure resource usage, express requests in terms of resources used, and extrapolate to the known limit.	1,000 requests/s with 16 CPUs at 40% gives $16 \times 40\% / 1,000 =$ 0.64% CPU per request , so $100\% \times 16 / 0.64 =$ 2,500 requests/s at saturation.	That is a rough best-case estimate : some other factor may bind first. It also used a single data point, and monitoring over time gives several at different rates, which improves the extrapolation. Then ask the separate question — is 2,500/s enough? A site averaging one request a second may take most of its 100,000 daily hits in the seconds after new content appears.
Factor analysis	Testing a limited set of configurations rather than all of them: measure at maximum, change one factor at a time, attribute a percentage drop and a cost saving to each, then compose the cheapest configuration that still meets the requirement — and retest it.	Eight binary factors would need 256 tests exhaustively; this approach needs about ten .	The retest at the end is not optional: components can behave differently in combination than their individual measurements predict. In the worked example, dropping to two processors and one network card gives $2 \times (1 - 0.30) \times (1 - 0.25) \approx$ 1.04 GB/s against a 1 GB/s requirement — a margin thin enough that confirming it matters.

Quantifying gains	Two ways. Observation: choose a reliable metric, estimate the gain from resolving the issue. Experiment: apply the fix and compare before with after.	Observed: a request takes 10 ms, of which 9 ms is disk I/O; caching in memory at around 10 μ s gives 10 ms $\rightarrow$ 1.01 ms, roughly a 9$\times$ gain . Experimental: 10 ms $\rightarrow$ 2 ms after raising concurrency, a 5 $\times$ gain.	Latency suits this because it can be compared directly across components. One caution: make sure it is measured as a synchronous part of the request — background work such as a write flush does not directly affect the request's time. Use observation rather than experiment when the fix is expensive to attempt in production.
Which mean	The arithmetic mean is the usual one. The geometric mean is the nth root of the product; the harmonic mean is the count divided by the sum of reciprocals.	Geometric suits improvements that compound multiplicatively — several layers of a stack each improved, all acting on the same packet. Harmonic suits rates : 800 MB, the first 100 at 50 MB/s and the rest at 10 MB/s, gives $800/(100/50 + 700/10) = \mathbf{11.1}$ MB/s.	vs reaching for the arithmetic mean by default, which for an average of rates is simply the wrong operation. Also note the decayed average , still over an interval but weighting recent time more heavily, which dampens short-term fluctuation — the familiar load averages work this way.
Averages over time	Nearly every performance metric is an average over some interval, and the interval is part of the number.	A CPU is never “at 50% utilization”; it was utilized during 50% of an interval — a second, a minute, an hour.	The failure this causes: a monitoring tool reporting five-minute averages never showed above 80%, while the real cause was CPU saturation during periods when utilization hit 100% for seconds at a time. Always check the interval before believing an average is reassuring.
Coefficient of variation	The ratio of the standard deviation to the mean, expressed as a single number.	A standard deviation of 50 tells you almost nothing on its own; a standard deviation of 50 with a mean of 200 tells you a great deal — a CoV of 25%.	Lower means less variance. A related single-figure expression is the z value , how many standard deviations a particular value sits from the mean.
Multimodal distribution	Two or more clusters. Means, standard deviations and percentiles are intended for normal-like or unimodal data.	Disk I/O latency for a mixed workload: one mode under 1 ms (on-disk cache hits) and one peaking near 7 ms (misses, random reads). The mean is 3.3 ms .	That 3.3 ms is not an index of central tendency — it is almost the opposite, sitting in the gap between the two things that actually happen. System performance is frequently bimodal: a fast code path and a slow one, or cache hits and cache misses. Every time you see an average latency, ask what the distribution is.
Outliers	A very small number of extreme values that do not fit the distribution, whether it is unimodal or not.	Occasional disk I/O taking over 1,000 ms when the majority are between 0 and 10 ms. Network latency outliers from timer-based retransmits are another.	They cause serious problems and are hard to see in most metric types other than a maximum. For a normal distribution their presence shifts the mean a little but not the median — so a mean and median that disagree is a signal. The standard deviation and the 99th percentile have a better chance of catching them, but that still depends on how often they occur.

The pictures that answer “what is the distribution?”

- **Line chart.** The familiar one: a metric against time. Adding lines for the median, standard deviation and percentiles to the same axes turns it from a summary into something informative — in the worked example the mean read latency sits around 4 ms, higher than expected, and the extra lines show why: the median is around 1 ms, where the latency was expected to be, and 1% of the I/O is over 20 ms.

- **Scatter plot.** Every event drawn as a point — completion time against latency. It shows *all* the data, which is how the outliers at 10, 20 and over 50 ms become visible at all. Its limits are resolution, where dense points overlap and merge into a wall of paint, and the volume of data that must be collected and processed.
- **Heat map.** Quantise both axes into buckets and colour each block by how many events fall in it. This solves both scatter-plot problems at once: it can show one system or thousands the same way, and the bulk patterns emerge instead of drowning. High-latency outliers appear as isolated blocks high in the map, and a bimodal distribution is immediately legible.
- **Timeline chart.** Activities as bars on a timeline — used for request waterfalls in browsers and for threads or CPUs on servers. With explicit dependency arrows it becomes a Gantt chart.
- **Surface plot.** Three dimensions rendered as a surface, best when the third value changes smoothly. Per-CPU utilization across a fleet: each server's CPUs as rows, per-second measurements as columns, height and colour set by the value. A server pinned at 100% appears as a plateau; a single CPU running constantly at a few percent appears as a faint line.

MONITORING IS WHAT GIVES A NUMBER ITS CONTEXT

Recording metrics over time lets the past be compared with the present, which is how you know what “normal” was. Expect cycles: **hourly** (reporting and monitoring tasks, often on five- or ten-minute cycles too), **daily** (work hours, stretched across time zones; plus nightly log rotation and backups), **weekly** (workdays against weekends), **quarterly** (financial reporting), and **yearly** (school terms and holidays). Irregular increases come from content releases and sales events; irregular *decreases* come from power and internet outages, and from everybody watching a sports final instead of using your product.

CONCEPT BOUNDARY — A SUMMARY STATISTIC IS A CLAIM ABOUT THE SHAPE OF THE DATA

A mean is an index of central tendency — but only if the data has a centre. For a bimodal distribution it can land precisely where nothing happens, which makes it not merely imprecise but actively misleading.

Percentiles are better and not sufficient. They quantify the slow tail, which is why they appear in service-level agreements, and the 50th — the median — shows where the bulk of the data sits. But they are still summaries of a distribution they assume is well behaved.

The only reliable move is to look at the full distribution: a histogram, a scatter plot, a heat map. Everything else is a claim you have not checked.

Anchor sketch — the models, and where the numbers deceive you

Legend: bold = a named node **reversed** = focus node Q = self-test cue

scalability profiles you can see without any model
linear . contention . coherence . knee point . ceiling

and two you can fit

AMDAHL $C(N) = N / (1 + a(N-1))$

a = the serial fraction. this IS contention.

USL $C(N) = N / (1 + a(N-1) + bN(N-1))$

b = coherency delay. with b = 0 it is Amdahl.

+--> fit a model so the DEVIATION can teach you:

either the model is wrong, or the system is.

QUEUEING THEORY

Little's Law $L = (\lambda)W$

Kendall $A/S/m$: arrivals / service / servers

M/D/1: BEYOND 60% utilization the mean response time
has DOUBLED. by 80%, TRIPLED. and that is the BEST
case -- percentiles degrade much faster.

+--> which is why disk utilization is a problem long
before 100%: its requests cannot be interrupted.
CPUs differ; higher priority preempts.

then the statistics turn on you

an AVERAGE is an index of central tendency -- only if
the distribution HAS a centre

MULTIMODAL a cache-hit mode under 1 ms and a miss
mode near 7 ms gives a mean of 3.3 ms, which
describes nothing that ever happened

OUTLIERS 1,000 ms among I/O of 0-10 ms. shifts the
mean a little; leaves the median alone.

THE INTERVAL a five-minute average hides seconds
spent at 100%, and seconds are enough to queue

+--> so every time you meet an average, ask:

WHAT IS THE DISTRIBUTION?

and the pictures that answer it

line chart + percentile lines . scatter plot (all the
data, until it becomes a wall of paint) . HEAT MAP
(quantised: one system or thousands) . timeline . surface

Q Give both scalability formulas and say what each extra parameter represents.

Q What has happened to mean response time beyond 60% utilization, and why does it matter more for disks than CPUs?

Q Why is a mean of 3.3 ms worse than useless for a bimodal distribution?

Trade-offs

DECISION	BUYS YOU	COSTS YOU	CHOOSE IT WHEN
Visual identification of a profile	Fast, needs no mathematics, and often enough to direct the next step.	No prediction, and no parameter you can compare between systems.	First, whenever you have enough data points to plot.

Fitting Amdahl or the USL	A parameter for seriality — and for coherency in the USL — plus predictions to test against reality.	Data collection across a range, regression, and the risk of trusting a model over the system.	When you need to predict, or when a deviation from the model would itself be informative.
Modelling instead of measuring	Answers about scales you cannot afford to build.	Large environments are complex and hard to model accurately, and a model contains only the limiters you thought of.	When renting the real thing briefly is not possible — otherwise test at scale and get measured knee points instead of theoretical ones.
Extrapolating a resource limit	A concrete ceiling in requests per second, from data you already have.	A best case: something else may bind first, and one data point extrapolates badly.	Capacity planning, using several data points over time rather than one.
Reporting an average	One number, universally understood, cheap to compute and to store.	It hides the distribution, and for bimodal data it can point at a value that never occurred.	Alongside the median, percentiles and the interval — never alone.
A heat map instead of a scatter plot	All the structure, at any scale, with outliers still visible as isolated blocks.	Quantisation loses individual events, and it is a heavier visualisation to produce.	Whenever the scatter plot has become a wall of paint, or the data spans many systems.

Say it so you understand it → say it like an expert

SAY IT THIS WAY TO <i>UNDERSTAND</i> IT	SAY IT THIS WAY TO SOUND LIKE AN <i>EXPERT</i>
“Average latency is 3 ms, that’s fine.”	“What’s the distribution? If it’s bimodal the mean sits between the modes and describes nothing that happened. Show me a histogram or a heat map.”
“The disk is only 60% utilized.”	“Past 60% its mean response time has already doubled. Disk requests can’t be preempted, so queueing bites long before 100% — unlike CPUs.”
“It never goes above 80%.”	“Over what interval? A five-minute average will hide seconds at 100%, and seconds are enough to produce saturation.”
“It stops scaling past eight threads.”	“That’s a knee point. Is it contention or coherence? Fit Amdahl and then the USL — if the coherency term is doing work, adding capacity will make it worse.”
“Standard deviation is 50.”	“Relative to what mean? On its own it says nothing. As a coefficient of variation it’s one number I can compare.”
“Average transfer rate across the two segments is 30 MB/s.”	“That’s the arithmetic mean of rates, which is the wrong operation. Harmonic mean — it works out at about 11.”
“We’ll model the cluster before we build it.”	“Or rent it for an afternoon. A model only contains the limiters we thought of; a real test at scale finds the ones we didn’t.”

PART 4 · QUESTION 1

A team reports that their storage is healthy: mean I/O latency is 3.2 ms against a 10 ms target, device utilization averages 55%, and the 99th percentile is 12 ms. Users of one feature report frequent multi-second stalls. The team's dashboard shows five-minute averages, and their conclusion is that the application must be at fault.

(a) Give two independent reasons the 3.2 ms mean may be consistent with users experiencing stalls, naming the phenomenon in each case. **(b)** Address the 55% utilization figure directly, with the queueing result and the reason it applies more forcefully to disks than to CPUs. **(c)** Say what you would ask for instead of these three numbers, and name the visualization you would use and why it beats the alternatives at this scale.

PART 4 · QUESTION 2

A service handles 2,000 requests/s. It runs on 24 CPUs averaging 45% utilization, and CPU is the busiest resource. Marketing expects a campaign to triple traffic. An architect proposes doubling the instance count and, separately, claims the system “scales linearly, so twice the machines means twice the throughput”. A previous scaling test showed throughput rising to a peak at eight instances and then *falling* as more were added.

(a) Compute the request rate at which CPU saturates, showing the arithmetic, and state two reasons the figure is optimistic. **(b)** Name the scalability profile the previous test shows, say which of the two models is needed to describe it and why, and say what that implies for the doubling proposal. **(c)** Say what you would measure before the campaign and name the method that would let you compose the cheapest configuration that meets the requirement.

2×2 — the shape of the data, and how much of it you looked at

COLUMNS: ARE YOU LOOKING AT A SUMMARY, OR AT THE FULL DISTRIBUTION? →

	A summary — mean, percentiles, deviation	The whole distribution
The data is UNIMODAL	The case the statistics were built for One cluster, a real centre, a tail you can quantify. <i>Move:</i> the mean is a genuine index of central tendency, the median tells you where the bulk sits, and percentiles quantify the slow end. Still check the <i>interval</i> — a five-minute average hides seconds at 100% whatever the shape.	Confirmation, and outlier hunting You looked, and it was one hump. <i>Move:</i> not wasted. This is how you learn the summary is trustworthy — and it is also the only reliable way to see outliers, which shift the mean only slightly and leave the median untouched.
The data is MULTIMODAL	The mean describes nothing that happened Sub-millisecond cache hits and ~7 ms misses give a mean of 3.3 ms — a value that sits in the gap between the two things the system actually does. <i>Move:</i> recognise that this is not imprecision, it is the opposite of central tendency. Percentiles help and do not rescue you, because they too assume a well-behaved shape. Any conclusion drawn here is about a system that does not exist.	Two answers, and now you can act The histogram or heat map shows the modes, and each one has a cause: a fast path and a slow path, or hits and misses. <i>Move:</i> stop trying to improve “the average” and start asking what moves traffic from the slow mode to the fast one — which is a different and far more tractable question.

Read it as: the column is a choice you make and the row is a fact you do not control — so the top-left cell is safe only by luck, and you cannot know you are in it without visiting the right-hand column at least once. That is the whole argument for looking at a distribution before trusting any number computed from it.

CARRY-OUT RULE FROM THIS PART

Ask for the distribution and the interval before you believe the number. An average with neither is a claim about the shape of data nobody has looked at — and in this field the shape is bimodal often enough that the claim is a coin flip.

Concept sketch — the whole competency on one page

Three named groups, in the order the story runs: **WORDS** → **POSTURE** → **METHOD & NUMBER**. Every node carries a question. Cover the answers, walk the map, and each answer hands you the next question.

Legend: bold = a group heading

reversed = the one node to remember

Q = retrieval cue

The three groups run in order: **WORDS** — what the terms commit you to · **POSTURE** — where you chose to stand · **METHOD & NUMBER** — what to run, and whether to believe it.

GROUP 1 - WORDS . the terms decide the measurement

LATENCY is the wait; RESPONSE TIME is wait + service + transfer -- the word covers both, so qualify it. and convert other metrics TO time: only time can be ranked and turned into a predicted speedup.

UTILIZATION has two definitions

time-based $U = B/T$ (what your tools report)
capacity-based (what fraction it COULD deliver)
-> 100% BUSY IS NOT 100% CAPACITY

SATURATION begins at 100% CAPACITY. any amount is an issue, because queueing is latency.

KNEE POINT: linear scaling turns into contention.
tune closest to the work (20x), not at the bottom (20%)

v

GROUP 2 - POSTURE . where you stood decides what you saw

RESOURCE ANALYSIS bottom-up | WORKLOAD ANALYSIS top-down
the documentation teaches the first. that is why people think it is the only one.

three ANTI-METHODS: streetlight . random change . blame someone else. the tell: NOTHING WAS OBSERVED FIRST.
(the ad hoc checklist is a real method that decays)

instead, in order:

PROBLEM STATEMENT six questions, no tools, first
SCIENTIFIC METHOD ruling things OUT is a result
DIAGNOSIS CYCLE get to data fast; kill bad theories

v

GROUP 3 - METHOD AND NUMBER . run one, then distrust it

USE every RESOURCE: utilization, saturation, errors
errors first, saturation next, utilization last.
iterate over resources, NOT tools -> unanswerable questions become recorded known-unknowns.

RED every SERVICE: rate, errors, duration
USE = machine health. RED = user health.

rate flat + duration up = architecture; both = load

WORKLOAD CHARACTERIZATION who . why . what . how changing

LATENCY ANALYSIS split in two, follow the larger half

STATIC TUNING check the config with NO load. skipped.

the numbers:

M/D/1 BEYOND 60% utilization -> response time DOUBLED

Amdahl a = seriality; USL adds b = coherency

a MEAN on bimodal data describes nothing real

-> always ask: what DISTRIBUTION, over what INTERVAL?

The question per node — cover the right column

GROUP 1 — WORDS

NODE	QUESTION	ANSWER IN ONE LINE
Latency vs response time	Define both, and say why it matters.	Latency is time spent waiting to be serviced; response time is that plus servicing plus transfer. The word is used for both, so name what you measured.
Converting to time	Why prefer latency over IOPS?	Time can be compared across components, ranked, and used to calculate a predicted speedup. IOPS can do none of those.
Utilization's two senses	Give both, and the example that splits them.	Busy for what fraction of the interval, versus what fraction of deliverable throughput is used. The lift runs constantly and still has room; so does a storage array with idle disks.
Saturation	Where does it start, and how bad is a little?	At 100% capacity-based utilization. Any amount is a performance issue, because it is queueing and queueing is latency.
Cache hit ratio vs miss rate	Why is the ratio not enough?	The ratio's effect is non-linear — 98% to 99% dwarfs 10% to 11% — and 90% at 200 misses/s can be worse than 80% at 20.
Where tuning pays	Application or device, and the caveat?	Closest to the work: factors at the application, percentages at the device. But the best place to <i>observe</i> from may still be the operating system.
When to stop	Give the three defensible reasons.	You have explained the bulk of the problem; the return is now below the cost of analysis; or there is a bigger return elsewhere.

GROUP 2 — POSTURE

NODE	QUESTION	ANSWER IN ONE LINE
Two perspectives	Name them and their metrics.	Resource analysis, bottom-up: IOPS, throughput, utilization, saturation. Workload analysis, top-down: requests, latency, completion — with throughput and latency as its metrics.
Streetlight	What is the specific danger?	You search where the light is. It may find an issue but not the issue, and nothing in the approach quantifies the finding.
Random change	What does it leave behind?	Tuning nobody understood, possibly working around a bug that is later fixed — and a change that can cause a worse problem at peak.
Blame someone else	How do you identify it, and what do you ask for?	A hypothesis with no data behind it. Ask for the screenshots showing which tools were run and how the output was read.
Reversed: problem statement	Why is this the node to remember?	Six questions, no tools, no access — and it should be first on every new issue, because it frequently identifies the cause outright.
Scientific method	Give the five steps and the discipline.	Question, hypothesis, prediction, test, analysis. Ruling things out is a result, and a failed experimental change is reversed before the next hypothesis.
Tools method's blind spot	What makes it dangerous rather than merely limited?	The view is bounded by the tools you have — and the user is unaware the view is incomplete, and may stay unaware.

GROUP 3 — METHOD AND NUMBER

NODE	QUESTION	ANSWER IN ONE LINE
USE	Three metrics, the order, and the reason.	Utilization, saturation, errors — checked errors first (objective, quick to rule out), saturation second (any amount matters), utilization last.
USE vs the tools method	What is the structural difference?	USE iterates over resources and then looks for tools, so an unanswerable question becomes a recorded known-unknown instead of an invisible gap.
RED	What does the request rate tell you?	Duration up with rate flat points at the architecture; both up means the problem may be one of load. That distinction is available before any deep analysis.
Workload characterization	The four questions, and why ask when you know?	Who, why, what, how changing. Because you may be wrong — the classic finding is that the clients are not who you assumed.
Static performance tuning	What makes it distinctive?	It runs with no load applied, checking the configured architecture rather than dynamic behaviour. Easy to do, easy to forget.
M/D/1 past 60%	What has happened, and why does it hit disks harder?	Mean response time has doubled; by 80% it has tripled. Disk requests cannot be interrupted, whereas higher-priority work can preempt on a CPU.
Multimodal	Why is this the number to distrust above all others?	A mean between two modes is not central tendency but nearly its opposite, so every summary computed from it describes a system that does not exist.

Master 2x2 — the decision card you can carry to other problems

Two questions separate an investigation from an opinion, and the second is the one people skip.

COLUMNS: DO YOU KNOW HOW BIG THE THING YOU FOUND IS? →

	No — you have not sized it	Yes — you can put a number on it
The CONCLUSION came first	A guess wearing a diagnosis You had a theory, and you have nothing that preceded it. <i>Move:</i> the honest question is what you observed <i>before</i> you believed this. If the answer is nothing, no amount of confidence or seniority changes the category — and the cost of acting on it usually lands on somebody else.	A number recruited to a prejudice You decided, then measured — so you measured the thing you already believed, and it agreed. <i>Move:</i> the most persuasive of the four cells and the hardest to escape. The test is whether you looked for anything that would have changed your mind, and whether the measurement can rule alternatives <i>out</i> rather than merely support this one.
The DATA came first	A finding you cannot rank Real, observed, and of unknown importance. <i>Move:</i> keep going, but do not act yet. Without a size you cannot tell a contributor from the cause, you cannot compare it with the other things competing for your attention, and — the part people miss — you have no principled way to decide when to stop looking.	Something you can act on Observed first, then quantified against the size of the whole problem. <i>Move:</i> now you can do the three things that matter. Rank it against everything else. Decide whether the return justifies more work. And stop — because you can say what fraction of the problem is explained, rather than stopping when you get tired or interested in something else.

The move that matters: the row protects you from being wrong; the column protects you from being busy. Most people who work carefully still live in the bottom-left — genuinely investigating, genuinely unable to say whether the thing they found matters — and that is where effort goes missing without anyone doing anything careless.

THE THUMB RULE, IN ONE SENTENCE

Get to data before you get to a conclusion, and put a number on what you find — the first stops you being wrong, the second stops you being busy.

It travels intact. A clinician who has a theory before an examination, an auditor who samples after deciding what the answer is, a team that finds a genuine defect and cannot say whether it explains the customer complaint — all four cells appear wherever people diagnose things. The bottom-left is the honest failure and by far the most common: careful work that never gets sized, so nobody can rank it, act on it, or stop.

Symptom → diagnosis → move

Cover the middle and right columns and work down the left.

Investigating

WHAT YOU OBSERVE	MOST LIKELY DIAGNOSIS	WHAT TO DO
Someone has a confident theory and no evidence.	An anti-method — the tell is a hypothesis with no data behind it.	Ask what was observed first. For a redirected ticket, ask for the screenshots and the interpretation, and take them for a second opinion.
Three changes were made at once and it improved.	Random change anti-method.	You cannot attribute the fix. Reverse changes one at a time; in future, reverse a failed change before forming the next hypothesis.
Nobody has spoken to the person reporting the problem.	The problem statement was skipped.	Six questions: why do you think there's a problem, has it ever been good, what changed, can it be expressed as latency, who else is affected, what is the environment.
The usual dashboard shows nothing, so the investigation stopped.	Streetlight — bounded by familiar tools.	Enumerate the questions first, then look for tools. Record every question you cannot answer as a known-unknown.
A runbook that used to work now misleads.	An ad hoc checklist is a point-in-time recommendation and has decayed.	Refresh it, and put a method that starts from questions in front of it.
A cause was found but the slowdown is unexplained.	An issue, not <i>the</i> issue — the finding has not been sized.	Quantify it against the whole. If a 3× slowdown is 12% explained, keep going; systems commonly have several causes.
Investigation has run for days with no end in sight.	No stopping rule was agreed.	Stop when the bulk is explained, when the return drops below the cost of analysis, or when a bigger return exists elsewhere.

Measuring

WHAT YOU OBSERVE	MOST LIKELY DIAGNOSIS	WHAT TO DO
A resource sits at 100% utilization and nothing is slow.	Time-based utilization on a resource that accepts parallel work.	Read it as percent busy. Check saturation — a wait queue is what distinguishes busy from full.
Users are affected but utilization never exceeds 80%.	The averaging interval is hiding short bursts at 100%.	Shorten the interval, or check saturation directly. Seconds at 100% are enough to produce queueing.
A queue exists but every resource looks lightly loaded.	A resource control or a software limit rather than a physical one.	Apply the same three questions to the cap: usage against limit, limit-imposed errors or throttling, allocation failures — even when the host is calm.
Request duration rose; request rate is flat.	An architecture problem — the service itself.	Investigate how the work is done: contention, a single-threaded path, a degraded component.
Request duration and request rate both rose.	Possibly a load problem — the rate is a clue, not a verdict.	Characterise the workload — who, why, what, how changing — then eliminate, throttle, or add capacity.
The architecture is sound and the system is still slow.	More load than it can reasonably handle.	Workload characterization. The best wins come from eliminating work that should not exist.
Errors are non-zero but everything “works”.	Recoverable failures — retries, or a failed device in a redundant pool.	Investigate anyway. They degrade performance while going unnoticed, which is why USE checks them first.

The metric you want does not exist.	A known-unknown.	Write it down. Enumerating what you cannot measure is a result, and it is exactly what a tool-first approach would have concealed.
Collecting the metric changed the measurement.	The observer effect — metrics cost CPU cycles to gather and store.	Budget for it, especially with tracing tools in a baseline, so as not to perturb production.
A cache benchmark shows almost no benefit.	The cache was cold for most of the run.	Compute the warm-up time from the fill rate before believing any short test.
Hit ratio improved and nothing got faster.	The miss <i>rate</i> is what costs you, and it may be unchanged.	Check misses per second, and compute runtime as hit rate $\times$ hit latency plus miss rate $\times$ miss latency.
Memory is consumed twice for the same data.	Double caching between two layers.	Cache as high in the stack as possible, and remove the redundant lower cache.
Nothing has been checked since deployment.	Static configuration faults, which no dynamic method reveals.	Static performance tuning: does each component make sense, suit the workload, was it auto-configured well, has it degraded?
“Is this number normal?” cannot be answered.	No baseline exists.	Start collecting baselines, including before and after changes. Failing that, summary-since-boot is coarse but better than nothing.

Believing the numbers

WHAT YOU OBSERVE	MOST LIKELY DIAGNOSIS	WHAT TO DO
Mean latency is fine; users report stalls.	A multimodal distribution, outliers, or both.	Look at the full distribution — histogram, scatter plot, heat map. A mean between two modes describes nothing that happened.
Mean and median disagree substantially.	Outliers, or a skewed distribution.	Outliers shift the mean and leave the median; investigate the tail with percentiles and by tracing individual events.
A scatter plot has become a solid block.	The visual density limit, plus the cost of collecting every point.	Switch to a heat map: quantise both axes into buckets, which scales from one system to thousands.
One request was very slow for no visible reason.	A latency outlier caused by preceding events.	Trace the sequence. An event at the tail of a queue is slow because of what was ahead of it.
Disk response time is far worse than the device spec.	Queueing — beyond 60% utilization the mean has already doubled.	Reduce utilization or the arrival rate. Do not expect a disk to behave like a CPU; its requests cannot be preempted.
Throughput peaks and then falls as you add capacity.	Coherency overhead beyond the knee point.	Fit the Universal Scalability Law; if the coherency term is doing real work, adding capacity makes it worse.
Throughput flattens but does not fall.	Contention — the serial fraction dominating.	Fit Amdahl's Law for the seriality parameter, and attack the serial component.
An average of transfer rates looks too high.	The arithmetic mean applied to rates.	Use the harmonic mean. For improvements that compound across layers, use the geometric mean.
A standard deviation was quoted with no mean.	Variance stated without its scale.	Ask for the coefficient of variation, which is a single comparable number.
A capacity estimate came from one data point.	An extrapolation with no error bars, and a best case at that.	Use several points collected over time, and remember some other factor may bind before the one you extrapolated.
Testing every configuration is infeasible.	Combinatorial explosion — eight binary factors is 256 tests.	Factor analysis: measure at maximum, vary one factor at a time, compose the cheapest configuration that meets the requirement, then retest it.

Reverse mapping — you see a technique, what is it there for?

The other direction. Use this when auditing an approach rather than diagnosing an incident: for each thing you find someone doing, say what it was buying and what it does *not* buy.

WHAT YOU FIND PEOPLE DOING	WHAT IT COUNTERS	WHAT IT DOES <i>NOT</i> COVER
Asking the six problem-statement questions	Investigating the wrong problem, and the cost of starting with tools.	Nothing that requires measurement. It narrows the search; it does not replace one.
The USE method	Systemic resource bottlenecks, fast — and blind spots, by naming what cannot be measured.	Caches, which improve under high utilization rather than degrading; and workload problems, which are RED's and workload characterization's job.
The RED method	User-visible service health, and the load-versus-architecture question via the request rate.	Which resource is responsible. It is USE's complement, not its replacement.
Workload characterization	Work that should not exist at all — the largest available win.	Anything about how well the work is performed once it has been accepted.
Latency analysis	Not knowing where the time went, by halving it repeatedly.	Load problems: it will faithfully attribute time to a resource that is simply being asked for too much.
Event tracing	Detail destroyed by summaries, including outliers and the events that caused them.	Cheapness. Collection has overhead, and the observer effect is real.
Static performance tuning	Configuration faults that every dynamic method will step straight past.	Anything about behaviour under load — by definition, since it runs with none applied.
Baseline statistics	“Is this normal?” for the many metrics no dashboard covers.	Problems that were already present when the baseline was taken.
Micro-benchmarking	Complexity — fewer factors, so the result is interpretable; and, for a network throughput test, external network bottlenecks.	Representativeness. The workload is artificial and may not be the question production is asking.
Percentiles in an agreement	The slow tail being hidden by an average.	Multimodality. Percentiles still assume a well-behaved distribution.
A heat map	The scatter plot's density limit and data volume, at any scale.	Individual events — quantisation is the price of the scale.
Fitting Amdahl or the USL	Guessing at a scalability profile, and gives a parameter you can compare.	Limiters you did not include. A model contains only what you thought of, which is why testing at scale finds more.
An auto-scaling policy on a utilization target	Load problems, automatically — capacity follows demand.	Architecture problems. A single-threaded bottleneck will not improve however many instances arrive.
The performance mantras	Reaching for hardware first, by putting six cheaper moves ahead of it.	Telling you <i>where</i> the work is. Characterise first, then apply the order.

Numbers worth remembering

NUMBER	WHAT IT IS	WHY IT MATTERS IN AN ARGUMENT
1 cycle = 1 second	The scaling device for time: a 0.3 ns CPU cycle stretched to one second, so every other latency can be felt rather than read.	Turns orders of magnitude into intuition, which is the only way most people ever grasp them.
3 s · 33 s · 6 min	On that scale: first-level cache, third-level cache, and main memory access.	Shows the memory hierarchy as the enormous gap it is, rather than as adjacent rows in a table.
9–90 hours · 1–12 months	On that scale: solid-state disk I/O (10–100 μ s) and rotational disk I/O (1–10 ms).	The single most persuasive way to explain why avoiding an I/O beats making one faster.
4 years · 19 years	On that scale: a round trip across the United States (40 ms) and to Australia (183 ms).	Puts network latency in the same frame as memory and disk, so the three can be argued about together.
32 millennia	On that scale: a physical system reboot (5 minutes).	The reason “just restart it” is a different category of action from everything else on the list.
20× vs 20%	Tuning at the application level against tuning at the storage device level.	The order-of-magnitude case for tuning closest to where the work is performed — factors versus percentages.
12% vs 66%	A finding worth 12% of CPU when a 3× slowdown needs explaining; 66% would have accounted for it.	The concrete form of “an issue, not the issue”, and the reason a finding has to be sized before you stop.
U = B/T	Time-based utilization: busy time over the observation period.	The formal definition behind “percent busy”, and the one your tools are actually reporting.
~30 combinations	The approximate number of resource-and-metric pairs a USE checklist produces for a server.	Sets expectations: it is a real piece of work, some entries are hard and some impossible — and the easy ones catch most issues.
Beyond 60% → doubled	M/D/1 mean response time against utilization: doubled beyond 60%, tripled by 80%, from $r = s(2 - \rho)/2(1 - \rho)$.	The number that ends “it's only 60% utilized”. And it is the best case — percentiles degrade much faster.
1 ms → 10 ms	An idle rotational disk's service time against the same disk under load.	Makes the queueing result concrete for the resource where it bites hardest.
L = λW	Little's Law: requests in the system equals arrival rate times time in the system.	The one identity worth memorising from queueing theory; it relates the three quantities you can usually measure.
α and β	Amdahl: $C(N) = N/(1 + \alpha(N-1))$. USL: $C(N) = N/(1 + \alpha(N-1) + \beta N(N-1))$.	α is seriality, β is coherency, and $\beta = 0$ collapses one into the other — which is the cleanest way to remember both.
0.64% per request	16 CPUs at 40% serving 1,000 requests/s, giving a projected ceiling of 2,500 requests/s.	A complete capacity extrapolation in two lines of arithmetic — and a reminder that it is a best case from one data point.
256 → 10 tests	Exhaustive testing of eight binary factors, against factor analysis.	The reason factor analysis exists, and the size of the saving.
1.04 GB/s	Approximately 2 GB/s maximum $\times (1 - 0.30) \times (1 - 0.25)$, after dropping to two processors and one network card, against a 1 GB/s requirement.	Shows the composition arithmetic — and a margin thin enough to prove why the final retest is not optional.
10 ms → 1.01 ms	A request with 9 ms of disk I/O, cached in memory at around 10 μ s: roughly a 9× gain.	The template for quantifying a fix by observation, before spending anything on trying it.

11.1 MB/s	800 MB, the first 100 at 50 MB/s and the rest at 10 MB/s: $800/(100/50 + 700/10)$.	The harmonic mean in one line, and proof that averaging rates arithmetically gives the wrong answer.
CoV 25%	A standard deviation of 50 against a mean of 200.	Shows why a deviation alone says nothing, and gives the single number that can be compared across metrics.
3.3 ms	The mean of a bimodal disk latency distribution with modes under 1 ms and near 7 ms.	The canonical example of a mean that is nearly the opposite of central tendency.
>1,000 ms	Occasional disk I/O outliers where the majority fall between 0 and 10 ms.	Why a maximum is sometimes the only metric that shows the thing hurting your users.
5-minute average	The monitoring interval that showed CPU never above 80% while the real cause was seconds at 100%.	The reason to ask for the interval before accepting any reassuring average.
>2 h and >10 h	Warm-up for a 128 GB memory cache and a 600 GB flash cache filling at 16 MB/s ($2,000 \text{ reads/s} \times 8 \text{ KB}$).	Settles any argument about a short benchmark: below these durations you measured a cold cache.
98% → 99% vs 10% → 11%	Two equal-sized increases in cache hit ratio with very unequal effects.	The non-linearity in one comparison — and the reason to quote the miss rate alongside the ratio.

Glossary — the terms you should be able to define cold

TERM	DEFINITION, AND THE LINE THAT SEPARATES IT FROM ITS NEIGHBOUR
IOPS / throughput	I/O operations per second / the rate of work, as data volume or operations depending on context. Neither can accurately be compared across components; time can.
Latency / response time	Time spent waiting to be serviced / the whole operation including service and transfer. The word “latency” is used for both, so qualify it.
Utilization, time-based	$U = B/T$, busy time over the interval. What tools report, and better named percent busy.
Utilization, capacity-based	The fraction of deliverable throughput in use. 100% busy is not 100% capacity.
Saturation	Work requested beyond what can be serviced; also called pressure. Begins at 100% capacity utilization; any amount is an issue.
Bottleneck	The resource limiting the performance of the system. Identifying and removing systemic ones is the core activity.
Workload	The input applied to the system. Distinct from the system's architecture, and the two produce the same symptom from different causes.
Knee point	Where one function becomes another — linear scaling becoming contention. Beyond it, more load can yield less work.
Profiling	Building a picture by sampling state at intervals. Coarse by design; how coarse depends on the sampling rate.
Cache hit ratio / miss rate	$\text{hits}/(\text{hits}+\text{misses})$, non-linear in effect / misses per second, proportional to the penalty and often easier to interpret.
Cold / warm / hot	Empty or useless / useful but not high-ratio / commonly requested with a high ratio. Warming can take hours.
Known-unknown	Something you know you have not checked. Recording one is a result; the unknown-unknown is the one you cannot record.
Observer effect	Metrics cost cycles to gather, perturbing what they measure. Not the uncertainty principle, which is about pairs of physical properties.
Resource / workload analysis	Bottom-up from the hardware / top-down from the requests. Two perspectives; documentation mostly teaches the first.
Streetlight anti-method	Using the tools you know because that is where the light is. Finds an issue, not necessarily the issue.

Problem statement	Six questions asked before any tool. The first method to attempt on any new issue.
Scientific method	Question, hypothesis, prediction, test, analysis. Ruling out is a result; reverse failed changes before the next hypothesis.
Diagnosis cycle	hypothesis → instrumentation → data → hypothesis. Emphasises reaching data fast so bad theories die early.
USE method	Every resource: utilization, saturation, errors — errors first. Iterates over resources, not tools.
RED method	Every service: rate, errors, duration. USE is machine health; RED is user health.
Workload characterization	Who, why, what, how changing. Aimed at the input, and the route to eliminating work entirely.
Drill-down / Five Whys	Monitor, identify, analyse / ask why about five times. Stopping at the first plausible answer is the failure mode.
Latency analysis	Split the time, follow the larger half, repeat. A binary search of latency.
Static performance tuning	Checking the configured architecture with no load applied. Easy to do; the hard part is remembering.
Micro-benchmarking	A simple artificial workload with few factors. Easier to understand than a macro-benchmark, and less representative.
Kendall's notation	A/S/m — arrival process, service time distribution, number of service centres. M/D/1 is Markovian arrivals with fixed service times.
Amdahl / USL	Models of contention / contention plus coherency. USL with $\beta = 0$ is Amdahl.
Coefficient of variation	Standard deviation over mean, as one number. A deviation quoted without its mean says almost nothing.
Multimodal	Two or more clusters, commonly a fast path and a slow one. Means and percentiles assume you do not have this.

Answer key

PART 1 · Q1 — THE ARRAY THAT IS “MAXED OUT”

(a) The dashboard is almost certainly reporting **time-based utilization** — $U = B/T$, the fraction of the interval the resource was busy, which a disk tool would more honestly call *percent busy*. That is not the same as **capacity-based utilization**, the fraction of deliverable throughput in use. Storage arrays frequently run at 100% precisely because **some** disk is busy the whole time while the array as a whole has plenty of idle disks and can accept more work. The everyday example is a lift: it may be moving between floors 100% of the time and still have room for passengers. 100% busy does not mean 100% capacity.

(b) The morning window can be real because **an average conceals its interval**. Five-minute averages mask short periods at 100%, and a short burst of high utilization is enough to cause **saturation** and a queue — the toll-plaza case, where a 40% daily average tells you nothing about whether anyone queued during rush hour. The metric that settles it is **saturation**, expressed as a wait-queue length or as time spent waiting, because unlike utilization it has no ambiguous form: any non-zero value is work that arrived and could not be serviced, which is latency someone experienced.

(c) Collect the **queue length** (and errors) for the array, at a much shorter interval than five minutes, over the 09:15–09:40 window — that combination distinguishes “busy but coping” from “out of capacity”, and it is the pair the utilization figure cannot separate. The missing queue-length metric is a **known-unknown**: you know the metric exists and you have not observed it. Recording it as such is a result rather than a gap — it is precisely the kind of thing a tool-first approach would have left as an invisible blind spot. Credit for noting that a purchase justified by a number that cannot distinguish the two cases is unjustified.

PART 1 · Q2 — THE CACHE, THE FORUM TUNABLE AND THE REDUNDANT WORK

(a) Two independent reasons. First, **the cache was almost certainly cold** for most of a ten-minute run: caches start cold and warm over time, and when the cache is large or the next tier slow, warming takes far longer than people expect — the worked case is a 128 GB memory cache filling at 16 MB/s and taking over two hours. A benchmark shorter than the warm-up measures the wrong thing. Second, **the hit ratio's effect is strongly non-linear**, so a partially warm cache sitting at a low ratio produces very little of the benefit a hot cache would: the difference between 98% and 99% is far larger than between 10% and 11%. Also creditable: the right figure to check is the **miss rate**, not the ratio, since the misses are what cost time.

(b) It is a **point-in-time recommendation**. A tunable found elsewhere may be inappropriate for this system and workload, may have been appropriate once and not now, or may be a temporary workaround for a bug already fixed in a later version — the comparison drawn is with taking someone else's prescription medicine. What it is good for is discovering **which parameters exist and have needed changing**; your job then becomes deciding whether and how they should be tuned here. Note also that you may still miss an important parameter simply because nobody else has needed to change it. If adopted, it should be stored in version control with the reason and date.

(c) Ranking: **the redundant decoding first**, then the cache, then the storage tunable. The principle is that **tuning is most effective closest to where the work is performed** — at the application level you can eliminate work entirely, for gains of the order of 20×, whereas tuning down at the storage device happens after the higher-level stack has already executed and yields percentages, of the order of 20%. Re-reading and re-decoding the same image is unnecessary work, and eliminating unnecessary work is the largest win available. The caveat: **the most effective level to tune is not necessarily the most effective level from which to observe** — the redundant decoding may be easiest to see from operating system tools showing file system and disk I/O, or time on-CPU, rather than from inside the application.

PART 2 · Q1 — THE CHECKOUT INCIDENT

(a) The first engineer is using the **streetlight anti-method** — opening the tools they know and looking for something odd — followed by the **blame-someone-else anti-method**, redirecting to a team they are not on. The second is using the **random change anti-method**. What all three share, and the identifying feature to name, is that **the hypothesis arrived before the data, or with no data behind it at all**. Note also the first engineer's specific failure: elevated disk activity may be an issue, but nothing in the approach quantified it, so there is no basis for believing it is *the* issue.

(b) The **problem statement** should have come first, since it is the method to attempt before any other on a new issue. Its six questions: what makes you think there is a performance problem; has this system ever performed well; what changed recently, in software, hardware or load; can the problem be expressed in terms of latency or runtime; does it affect other people or applications; and what is the environment, with software, hardware, versions and configuration. It costs almost nothing because it requires **no tools, no access and no metrics** — issues have been resolved this way over the phone without logging in to any server. In this scenario the third question alone would likely have identified a recent change, and nobody has yet asked it.

(c) Two problems. First, **changing three values at once destroys attribution**: they cannot know which change mattered, and the discipline is to reverse a change that does not work before forming the next hypothesis, precisely because changing multiple factors at once makes it impossible to identify which one was responsible. Second, the one that appears later: they have left behind **tuning nobody understood in the first place**. If it happens to work around a bug in the database or operating system that is later fixed, the tuning remains, no longer makes sense, and nobody will dare remove it. Also creditable: a poorly understood change can cause a worse problem at peak production load, requiring an emergency rollback.

PART 2 · Q2 — THE 30-ENTRY CHECKLIST

(a) The checklist is genuinely valuable: an **ad hoc checklist** can provide the most value in the shortest time frame, and it is an effective way to ensure a whole team knows how to check for common issues, written clearly and prescriptively. Its two structural limitations: every entry is a **point-in-time recommendation** that needs frequent refreshing to stay current — which explains the tunable that made things worse — and checklists **skew toward issues with known, easily documented fixes**, typically tunable settings, rather than custom fixes to source code or the environment. That explains the two silent failures.

(b) Adding tools deepens the **tools method**, which relies exclusively on the tools you have or know and therefore gives an incomplete view of the system. The specific blind spot, and the reason it is dangerous rather than merely limited, is that **the user is unaware that the view is incomplete — and may remain unaware**. Issues requiring custom tooling may never be identified. It also scales badly: with many tools of overlapping functionality, time goes into evaluating which to use rather than into the investigation.

(c) Put the **problem statement** first on every issue, then a hypothesis-driven loop — the **scientific method** (question, hypothesis, prediction, test, analysis) or the tighter **diagnosis cycle** (hypothesis → instrumentation → data → hypothesis), moving to data quickly so bad theories are discarded early. Where a resource sweep is wanted, prefer the **USE method**, because it iterates over resources rather than tools: you build the complete list of questions first and only then look for something that can answer them. And the answer to the last part: a question you cannot answer should be **recorded as a known-unknown**. Knowing which questions are unanswered is extremely useful, and it is exactly the information a tool-first approach conceals.

PART 3 · Q1 — THE PAYMENTS SERVICE THAT SLOWED OVER THREE WEEKS

(a) Duration has risen while the **request rate is unchanged**, which points at **the architecture — the service itself** — rather than at load. That is the distinction the request rate buys you: if both rate and duration had risen, the problem might be the load applied, and the investigation would go to workload characterization. Here it rules that out early, before any deep analysis.

(b) Run the **USE method**: for every resource, check **utilization, saturation and errors**. The order is **errors first**, because they are usually objective rather than subjective and so are quick to interpret and time-efficient to rule out; **saturation second**, because it is quicker to interpret than utilization and *any* level of it can be an issue; **utilization last**. For a capped container, apply the same three questions to the **resource controls** rather than only to the physical hardware: memory capacity utilization is the tenant's usage against its cap, and memory capacity saturation shows up as limit-imposed allocation errors or swapping **for that tenant even when the host system is not experiencing memory pressure**. That is exactly why the calm host dashboards are not reassuring — they are measuring the wrong boundary.

(c) **Static performance tuning**: check the configured architecture with no load applied, asking of each component whether it makes sense, whether its configuration suits the intended workload, whether it was auto-configured into the best state, and whether it has degraded after an error. Eleven months without review is exactly the situation it exists for — a negotiated-down network interface, a failed disk in a redundant pool, an old version, a nearly-full file system, a debug mode accidentally left enabled. These are easy to check for; the hard part is remembering to do it. Also creditable: **baseline statistics**, or failing that summary-since-boot values, to establish whether current figures are normal for this server.

PART 3 · Q2 — THE REPORTING DATABASE AND THE HANGING QUERIES

(a) **Workload characterization**, which focuses on the **input** to the system rather than the resulting performance. Its four questions: **who** is causing the load (process, user, remote address), **why** is it being called (code path, stack trace), **what** are its characteristics (rate, throughput, direction, type — with variance where appropriate), and **how** is it changing over time (is there a daily pattern). They should be asked even when the team is confident because **you may be surprised**: the worked example is a team who checked the client addresses of a database they expected to be used by their web servers, and discovered the entire internet was sending load — a denial-of-service attack. A weekday-morning pattern is itself a strong hint that the fourth question matters here.

(b) **Event tracing**. Summaries such as an average latency destroy exactly the detail that matters when a small number of operations behave very differently from the rest; tracing inspects events individually, recording for each the **input** (type, direction, size), the **times** (start, end, latency) and the **result** (error status, transfer size). It also surfaces something no summary can: a **latency outlier may be caused by preceding events rather than by its own properties** — an event at the tail of a queue is slow because of what was queued ahead of it. Also creditable: the distribution is probably multimodal, so the average is not an index of central tendency, and a histogram or heat map would show the modes.

(c) Before adding capacity you must establish whether this is a problem of **load or of architecture** — adding instances is the correct response to load, and does nothing for a single-threaded bottleneck or lock contention. The ordering principle is the **performance mantras**, most effective first: **don't do it** (eliminate unnecessary work); **do it but don't do it again** (caching); **do it less** (reduce refresh or polling frequency); **do it later** (write-back); **do it when they're not looking** (schedule off-peak); **do it concurrently**; and only last, **do it more cheaply** — buy faster hardware. Doubling the instances is the last item on that list, being proposed first. For a weekday-morning peak, “do it when they're not looking” is a particularly live option.

PART 4 · Q1 — THE STORAGE THAT IS “HEALTHY”

(a) Two independent reasons. First, a **multimodal distribution**: system performance is frequently bimodal — a fast path and a slow one, or on-disk cache hits and misses — and the mean then falls in the gap between the modes. In the worked case, modes under 1 ms and near 7 ms give a mean of 3.3 ms, which **is not an index of central tendency; it is almost the opposite**, and describes nothing that actually happened. Second, **outliers**: a very small number of extreme values, such as occasional disk I/O over 1,000 ms among a majority between 0 and 10 ms. These barely move the mean, do not move the median at all, and are exactly what a user experiences as a multi-second stall. Whether the 99th percentile catches them depends on how frequent they are; a maximum may be the only summary that shows them.

(b) 55% utilization is not the reassurance they think. Modelling the disk as **M/D/1** gives $r = s(2 - \rho)/(2(1 - \rho))$, and **beyond 60% utilization the mean response time has doubled; by 80% it has tripled**. So they are already close to the point where queueing dominates, and this is the **best case** — variation in service time drives the mean higher still, and the 90th and 99th percentiles degrade much faster than the mean past 60%. It applies more forcefully to disks than to CPUs because a disk is a queueing system whose requests **typically cannot be interrupted and must wait their turn**, whereas on a CPU higher-priority work can preempt. Add that the 55% is itself a five-minute average, which hides shorter periods at much higher utilization.

(c) Ask for **the full distribution and a shorter interval** rather than three summary numbers — and specifically for saturation (queue length) alongside utilization. The visualization is a **heat map**: it quantises both axes into buckets and colours by event count, which solves the two problems of a scatter plot at this scale — the visual density limit, where dense points merge into a wall of paint, and the volume of data that must be collected and processed for every event. It shows one system or thousands the same way, makes a bimodal distribution immediately legible, and leaves high-latency outliers visible as isolated blocks high in the map.

PART 4 · Q2 — THE CAMPAIGN AND THE SCALING CLAIM

(a) CPU per request = total CPU% ÷ requests = $24 \times 45\% \div 2,000 = 1,080 \div 2,000 = 0.54\%$ **CPU per request**. Maximum requests/s = $100\% \times 24 \text{ CPUs} \div 0.54 = 2,400 \div 0.54 \approx 4,440$ **requests/s**. Two reasons it is optimistic: it is a **rough best-case estimate** — some other limiting factor may be encountered before requests reach that rate — and it rests on a **single data point**, one throughput figure against one utilization figure, where monitoring over time would give several pairs and a much better extrapolation. Also creditable: long before 100% the queueing effects of the previous question apply; and a tripling of traffic to 6,000 requests/s exceeds the estimate by a wide margin anyway.

(b) Throughput rising to a peak and then **falling** as instances are added is the **coherence** profile — the tax of maintaining coherency, including propagating changes, has begun to outweigh the benefit of scaling. Amdahl's Law of Scalability models **contention** only, $C(N) = N/(1 + \alpha(N-1))$, which flattens but does not decline; you need the **Universal Scalability Law**, $C(N) = N/(1 + \alpha(N-1) + \beta N(N-1))$, whose β term is coherency delay — and note that with $\beta = 0$ the USL becomes Amdahl. The implication for the proposal is direct and uncomfortable: the system does not scale linearly, and past the peak **adding instances reduces throughput**, so doubling the count may deliver less capacity than they have now, not twice as much.

(c) Measure across a **range of N** — by observation over time or experimentally with micro-benchmarking or a load generator — and fit the model by regression to obtain α and β , then plot the data against the model, because a deviation is itself informative: either the model is wrong or your understanding of the system's real scalability is. Also collect multiple throughput-and-utilization pairs to firm up the resource-limit extrapolation. The method for composing the cheapest configuration that still meets the requirement is **factor analysis**: test at maximum, change one factor at a time and attribute a percentage drop and a cost saving to each, then choose the combination that meets the target — eight binary factors need about ten tests rather than 256 — and **retest the chosen configuration**, since components can behave differently together than their individual measurements predict.

HOW TO SCORE YOURSELF

Two marks, both quick. **Completeness**: did you name the method, and — for this competency especially — did you say *which situation it is the answer to*? Naming USE is a quarter of an answer; saying why errors come first, and what the method does that a tool-first sweep cannot, is the rest of it. **Phrasing**: could you have said it in an incident review without hand-waving? If you wrote “the average is misleading” where the key says “the distribution is bimodal, so the mean sits between the two things the system actually does and describes neither”, you have the label and not the mechanism — and in a room full of people looking at a dashboard, the mechanism is the only thing that moves anyone.

Spaced-review tracker

Review at **day 1, 3, 7, 16, 35**. Write the date in the box when you pass. On a fail, reset that row to a 1-day interval and start again.

ITEM	DAY 1	DAY 3	DAY 7	DAY 16	DAY 35	FAILS
1 • Latency vs response time, and why qualify it						
2 • Why convert other metrics to time						
3 • Utilization's two definitions, and the lift						
4 • Saturation: where it starts, why any is bad						
5 • Knee point, and scaling past it						
6 • Load vs architecture, with an example of each						
7 • Hit ratio vs miss rate, and cache warmth						
8 • Where tuning pays, and where to observe from						
9 • The three reasons to stop analysis						
10 • Known-knowns, known- and unknown-unknowns						
11 • Resource vs workload analysis						
12 • The three anti-methods, their tell, and the checklist						
13 • The six problem-statement questions						
14 • Scientific method, and the negative test						
15 • The tools method's invisible boundary						
16 • USE: three metrics, the order, the reason						
17 • USE on software resources and caps						
18 • RED, and what the request rate reveals						
19 • Workload characterization's four questions						
20 • Drill-down's three stages, and Five Whys						
21 • Latency analysis as a binary search						
22 • Event tracing, and outliers caused by predecessors						
23 • Static performance tuning's four questions						
24 • Cache tuning order, and double caching						
25 • The seven performance mantras, in order						
26 • The five scalability profiles						
27 • Amdahl and the USL, and what α and β mean						
28 • M/D/1 past 60%, and why disks suffer more						
29 • Resource limit extrapolation, and factor analysis						
30 • Which mean: arithmetic, geometric, harmonic						
31 • Multimodal distributions and outliers						

Final quiz — no notes, twenty minutes, write the answers

1. Give both definitions of utilization, the example that separates them, and say which metric decides whether a busy resource is actually a bottleneck.

.....

2. Explain why converting a performance metric to latency is worth doing, and name two things it makes possible that a rate metric does not.

.....

3. Name the three anti-methods, give the feature they all share, say why the ad hoc checklist is not one of them, and say what you should ask someone who has redirected an issue to another team.

.....

4. Give the six problem-statement questions and say why this method should come before any tool.

.....

5. State the USE method, give its three metrics in the order they should be checked with the reason for that order, and say what it does differently from the tools method.

.....

6. State the RED method and explain precisely how the request rate distinguishes a load problem from an architecture problem.

.....

7. Give workload characterization's four questions, and say why they should be asked even when you are confident of the answers.

.....

8. Name the method that runs with no load applied, give its four questions, and say why it is the one people skip.

.....

9. Give both scalability formulas, say what each parameter represents, and state what has happened to mean response time beyond 60% utilization under M/D/1 — and why that matters more for disks than for CPUs.

.....

10. Explain why a mean can be worse than useless for a bimodal distribution, name two other statistics that also fail there, and give the only reliable alternative.

.....